A Project Report
On

Eduke: An Academic Performance Prediction System

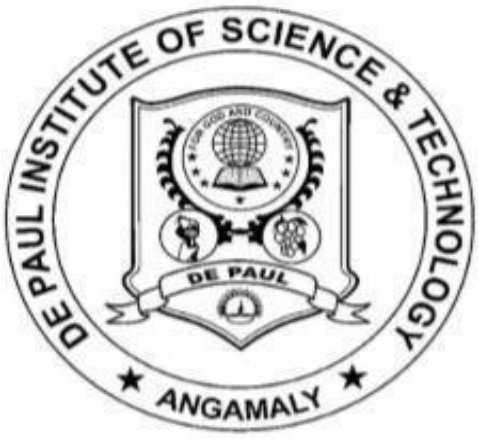
Submitted to the
Department of MCA

In partial fulfillment of the

MASTER OF COMPUTER APPLICATIONS

Under guidance of
Asst. Prof. Soumya M

Project done by
SAIN SABURAJ
Reg No: 233242210382

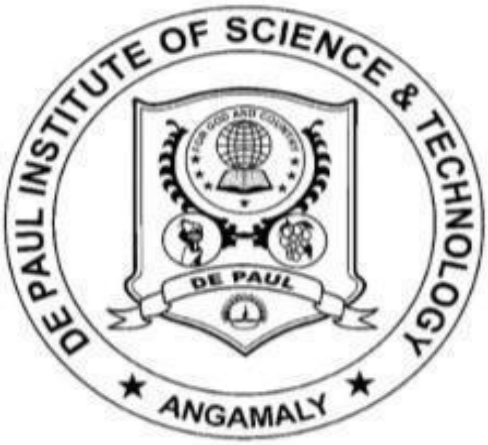


DEPARTMENT OF MCA

DE PAUL INSTITUTE OF SCIENCE & TECHNOLOGY (DiST)
ANGAMALY SOUTH, KERALA

(Affiliated to Mahatma Gandhi University, Kottayam)
MARCH 2025

DE PAUL INSTITUTE OF SCIENCE & TECHNOLOGY
(DiST) ANGAMALY SOUTH, KERALA
(Affiliated to Mahatma Gandhi University, Kottayam)



BONAFIDE CERTIFICATE

Certified that the Project Work entitled

"Eduke: An Academic Performance Prediction System"

is a bonafide work done by

SAIN SABURAJ

In partial fulfillment of the requirement for the Award of

MASTER OF COMPUTER APPLICATIONS
Degree From

Mahatma Gandhi University, Kottayam
2023 - 2025

Head of the Department **Project Guide**

Submitted for the Viva-Voce Examination held on.....

External Examiner1 **External Examiner 2**

DE PAUL INSTITUTE OF SCIENCE & TECHNOLOGY (DiST)
ANGAMALY SOUTH, KERALA
(Affiliated to Mahatma Gandhi University, Kottayam)

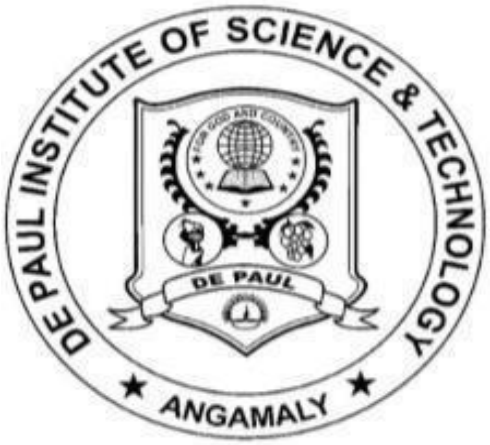
CERTIFICATE

This is to certify that the project entitled "Eduke: An Academic Performance Prediction System" has been successfully carried out by SAIN SABURAJ (Reg. No: 233242210382) in partial fulfilment of the Course Master of Computer Applications.

INTERNAL GUIDE

DATE: **HEAD OF THE DEPARTMENT**

DE PAUL INSTITUTE OF SCIENCE & TECHNOLOGY
(DiST) ANGAMALY SOUTH, KERALA
(Affiliated to Mahatma Gandhi University, Kottayam)



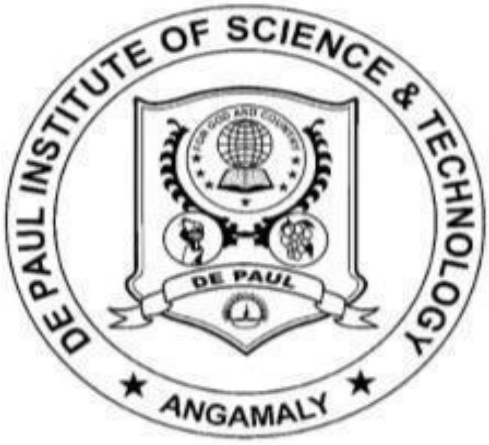
CERTIFICATE

This is to certify that the project entitled "Eduke: An Academic Performance Prediction System" has been successfully carried out by SAIN SABURAJ (Reg no: 233242210382) in partial fulfilment of the course Master of Computer Applications under my guidance.

DATE: **Asst. Prof. Soumya M**

INTERNAL GUIDE

DE PAUL INSTITUTE OF SCIENCE & TECHNOLOGY
(DiST) ANGAMALY SOUTH, KERALA
(Affiliated to Mahatma Gandhi University, Kottayam)



DECLARATION

I, SAIN SABURAJ, hereby declare that the project work entitled "Eduke: An Academic Performance Prediction System" is an authenticated work carried out by me under the guidance of Asst. Prof. Soumya M for the partial fulfilment of the course MASTER OF COMPUTER APPLICATIONS. This work has not been submitted for similar purpose anywhere else except to DE PAUL INSTITUTE OF SCIENCE & TECHNOLOGY. I understand that detection of any such copying is liable to be punished in any way the school deems fit.

Date: **SAIN SABURAJ**

Place:

Signature

ACKNOWLEDGEMENT

First and foremost, praises and thanks to the God, the Almighty, for His showers of blessings throughout my project work to complete the project successfully. I respect and thank **Fr. (Dr). Johny Chacko Mangalath V C**, Our Principal of De Paul Institute of Science & Technology (DiST) Angamaly giving me all necessary facilities in the college for successful completion of this dissertation. I record my gratitude to **Assoc. Prof. Jacob Thaliyan HOD**, School of Computer Science (PG) for his support and guidance which made me complete the project duly. I am extremely thankful to him for providing such a nice support and guidance, although he had busy schedule. I owe my deep gratitude to **Asst. Prof. Soumya**, Guide of this project who took keen interest on my project work and guided me all along, till the completion of my project work by providing all the necessary information for developing a good system. I hereby express my sincere thanks to **Asst. Prof. Dinumol Philip** class in charge, for the kind cooperation and valuable suggestions throughout the course of my project. I am thankful to and fortunate enough to get constant encouragement, support and guidance from all teaching staffs of SCHOOL OF COMPUTER SCIENCE which helped me in successfully completing my research work. I am thankful to my family and friends who were a source of inspiration for the successful completion of this project work. Finally, yet importantly, my thanks go to all the people who have supported me to complete the project directly or indirectly.

SAIN SABURAJ

TABLE OF CONTENTS

1. Introduction.....Page 6
   - 1.1 Introduction.....Page 7
   - 1.2 Objectives.....Page 7
   - 1.3 Scope and Relevance of the Project.....Page 8
2. System Analysis.....Page 10
   - 2.1 Introduction to System Analysis.....Page 11
   - 2.2 Existing System.....Page 11
     - 2.2.1 Limitations of Existing system.....Page11
   - 2.3 Proposed System.....Page 12
     - 2.3.1 Advantages of the Proposed System.....Page 12
   - 2.4 Feasibility Study.....Page 13
     - 2.4.1 Technical Feasibility.....Page 13
     - 2.4.2 Operational Feasibility.....Page 13
     - 2.4.3 Economic Feasibility…….....Page 14
3. System Design.....Page 15
   - 3.1 Introduction to System Design.....Page 16
   - 3.2 Database Design.....Page 16
   - 3.3 Entity-Relationship Model.....Page 24
     - 3.3.1 Data Dictionary.....Page 25
   - 3.4 Object-Oriented Design – UML Diagrams.....Page 26
     - 3.4.1 Use Case Diagram.....Page 31
     - 3.4.2 Activity Diagram.....Page 32
     - 3.4.3 Class Diagram.....Page 43
     - 3.4.4 Sequence Diagram.....Page 44
   - 3.5 Modular Design….....Page 49
     - 3.5.1 Structure Chart.....Page 49
     - 3.5.2 Modules Description.....Page 50
   - 3.6 Input Design.....Page 51
   - 3.7 Output Design.....Page 55

4. System Environment.....Page 59
- 4.1 Introduction to System Environment.....Page 60
- 4.2 Software Requirements.….....Page 62
- 4.3 Hardware Requirements.....Page 62
- 4.4 Tools and Platforms.....Page 63
  - 4.4.1 Front End Tool.....Page 63
  - 4.4.2 Back End Tool.....Page 64
  - 4.4.3 Operating System.....Page 65

5. System Implementation.....Page 67
- 5.1 Introduction to System Implementation….....Page 68
- 5.2 Coding.....Page 70
  - 5.2.1 Coding Standards.....Page 70
  - 5.2.2 Sample Codes.….....Page 73
- 5.3 Debugging.....Page 88
- 5.4 Unit Testing.....Page 89

6. System Planning and Scheduling.....Page 91
- 6.1 Introduction to System Planning and Scheduling.....Page 92
  - 6.1.1 Planning A Software Project.....Page 92
    - 6.1.1.1 Steps Involved in Planning a System.....Page 92

7. System Cost Estimation.....Page 95
- 7.1 Introduction.....Page 96
- 7.2 Loc based Estimation.....Page 96

8. System Testing.....Page 98
- 8.1 Introduction to System Testing.....Page 99
- 8.2 Integration Testing.....Page 100
- 8.3 Validation testing.....Page 100
  - 8.3.1 Test Cases.....Page 100

9. System Maintenance.....Page 104
- 9.1 Introduction For System Maintenance.....Page 105
- 9.2 Maintenance.....Page 105

10. System Security Measures.....Page 106
- 10.1 Introduction To System Security Measures.....Page 107

- 10.2 Operating System Level Security.....Page 107
- 10.3 Database Level Security.....Page 108
- 10.4 System Level Security.....Page 109

11. Future Enhancement & Scope of Further Development.....Page 110
    - 11.1 Introduction.....Page 111
    - 11.2 Merits of the System.....Page 111
    - 11.3 Limitations of the System.....Page 112
    - 11.4 Future Enhancement of the System.....Page 113
12. References.....Page 115

ABSTRACT

The Eduke system is an innovative academic performance prediction and management platform, with its key feature being mark prediction. By leveraging predictive analytics, Eduke analyzes student attendance, study habits, past marks, and other academic factors to forecast future performance. This enables students to identify weak areas, set realistic academic goals, and take proactive measures to improve their scores. The platform provides a structured academic management system, allowing teachers and subject heads to efficiently record attendance, update marks, and track student progress. Parents can monitor their child's predicted performance and receive insights into their learning patterns, fostering better academic support at home. Eduke also integrates Eduke Bot, an intelligent chatbot designed to assist students in understanding their predicted marks. Using natural language processing (NLP) and intent classification, Eduke Bot provides personalized study recommendations, subject-specific guidance, and motivational support to help students improve their academic outcomes. Additionally, the system streamlines marks management, attendance tracking, and quiz evaluations, ensuring a data-driven and student-focused learning environment. With AI-powered mark prediction at its core, Eduke transforms academic management into a proactive and personalized experience, helping institutions, students, and parents make informed decisions for better educational outcomes.

INTRODUCTION

1.1 INTRODUCTION

The Eduke system is designed to revolutionize academic performance prediction and management by utilizing AI-powered mark prediction. The primary goal of this project is to help students, teachers, and parents understand and improve academic outcomes through data-driven insights.

Students often struggle with identifying their strengths and weaknesses before exams. Eduke's mark prediction feature analyzes attendance, past marks, study habits, and other academic factors to forecast future scores. This allows students to take proactive measures to enhance their learning.

The system provides an interactive platform where teachers and subject heads can efficiently record attendance, update marks, and track student progress. Additionally, parents can monitor their child's predicted performance and receive insights into their learning patterns, ensuring better academic support.

A key component of Eduke is Eduke AI, an intelligent chatbot that serves as a personalized academic assistant. Using natural language processing (NLP) and intent classification, it provides study recommendations, subject-specific guidance, and motivational support based on the predicted marks.

Moreover, Eduke integrates essential academic functions such as marks management, attendance tracking, and quiz evaluations into a seamless and structured system. By offering personalized predictions and AI-driven guidance, Eduke empowers students to improve their performance, teachers to make data-informed decisions, and parents to stay actively involved in their child's education.

1.2 OBJECTIVES

The main objective of this system is to enhance academic performance through predictive analytics by leveraging data-driven decision-making. However, students often struggle to identify their weak areas, track progress, and receive personalized guidance. To address these challenges, this project aims to develop an AI-powered academic performance prediction and management system, Eduke.

Eduke provides an advanced mark prediction feature that analyzes attendance, study habits, past performance, and other academic factors to forecast students' future marks. This enables students to take proactive steps toward improving their learning outcomes.

1.3 SCOPE AND RELEVANCE OF THE PROJECT

Eduke is an AI-powered academic performance prediction and management system designed to enhance student learning by forecasting their future marks based on attendance, study habits, and past academic performance. By leveraging predictive analytics, Eduke provides data-driven insights to help students improve their academic outcomes while assisting teachers and parents in monitoring progress effectively.

Scope of the Project

- Mark Prediction and Academic Insights: Eduke analyzes various academic factors to predict students' future marks, helping them identify areas for improvement and take proactive steps to enhance their learning.
- Student Dashboard: A user-friendly interface where students can track their predicted scores, view academic progress, and receive personalized study recommendations.
- Parental Monitoring System: Parents can monitor their child's predicted performance, ensuring they stay informed and engaged in their education.
- Eduke AI Chatbot: An intelligent assistant that provides students with subject-specific guidance, motivational feedback, and personalized study strategies based on their predicted performance.
- Attendance and Marks Management: Teachers can record attendance, update marks, and analyze student progress efficiently.

- Quiz Evaluations and Performance Tracking: Eduke includes features for conducting quizzes and tracking student responses to provide a comprehensive academic monitoring system.

Relevance of the Project

- Enhancing Student Learning: By predicting marks and providing insights, Eduke empowers students to take control of their academic progress.
- Data-Driven Academic Support: Teachers and parents can use predicted performance data to guide students effectively.
- Technological Advancement in Education: The project integrates machine learning, natural language processing (NLP), and data analytics to create an intelligent academic support system.
- Scalability and Future Expansion: Eduke has the potential to expand beyond mark prediction, integrating more AI-driven tools for student assessment, career guidance, and learning recommendations.
- By providing a personalized, predictive, and AI-enhanced learning experience, Eduke ensures that students receive actionable insights to improve their academic performance, making education more effective and student-centric.

SYSTEM ANALYSIS

2.1 INTRODUCTION TO SYSTEM ANALYSIS

System analysis is a critical phase in the development lifecycle of any complex system, software application, or technology solution. It involves a systematic approach to understanding, defining, and specifying the requirements, functionalities, and constraints of the system to be developed. System analysis serves as the foundation for the subsequent phases of system design, implementation, testing, and maintenance, guiding the development process and ensuring that the resulting system meets the needs and expectations of stakeholders.

2.2 EXISTING SYSTEM

- The current academic performance tracking systems in educational institutions primarily rely on manual assessment methods and historical performance reviews. Teachers evaluate student progress based on exam scores, assignments, and attendance, but there is no predictive mechanism to forecast future academic performance.
- Students often struggle to identify their weaknesses early, as there is no system in place to analyze past academic trends and provide actionable insights. Additionally, parental involvement is limited since most institutions provide performance reports only after exams, leaving little room for timely interventions.
- Moreover, existing systems lack AI-driven assistance, meaning students do not receive personalized study recommendations or interactive learning support. Academic tracking is mostly reactive rather than proactive, making it difficult for students to take early measures to improve their performance.
- Overall, the absence of predictive analytics, real-time performance monitoring, and AI-driven academic assistance results in an inefficient system that does not fully support students, teachers, and parents in optimizing learning outcomes.

2.2.1 LIMITATIONS OF EXISTING SYSTEM

- The accuracy of predictions generated by the Eduke system may be affected by data limitations, such as incomplete academic records, inconsistent attendance tracking, or variations in student study habits. As a result, the system's forecasts

may not always fully reflect a student's actual performance.

- The system is not fully optimized for all devices, which may lead to usability challenges on certain screen sizes and resolutions. Users accessing Eduke from smaller screens or older devices may experience layout distortions or difficulties in navigation, impacting their overall experience.
- Currently, the system does not offer personalized study recommendations tailored to each student's strengths and weaknesses. While it provides academic performance predictions, it lacks adaptive learning features that could suggest specific study strategies or resources to help students improve in weaker areas.
- The chatbot integrated into the Eduke system may not always be able to provide responses to every user query. Since it relies on predefined datasets and AI training models, it may struggle with complex, ambiguous, or highly specific academic questions, limiting its effectiveness as a comprehensive virtual assistant.

2.3 PROPOSED SYSTEM

- The proposed Eduke system introduces an AI-driven academic performance prediction and management platform designed to assist students, teachers, and parents. By analyzing student marks, attendance, and study habits, the system provides predictive insights into academic performance, helping students take proactive steps for improvement.
- The system includes a personalized recommendation feature that suggests study strategies based on individual learning patterns. An AI-powered chatbot, Eduke AI, enhances student engagement by offering academic guidance and answering queries dynamically.
- Additionally, the platform ensures better parental involvement by providing real-time access to student performance data. With a user-friendly and responsive interface, the system aims to make academic tracking more efficient, interactive, and data-driven, ultimately helping students achieve better learning outcomes.

2.3.1 ADVANTAGES OF THE PROPOSED SYSTEM

- Improved academic performance prediction through AI-driven analysis
- Personalized study recommendations based on student learning patterns
- Increased parental involvement with real-time access to student progress
- AI-powered chatbot providing academic guidance and instant query resolution
- Proactive identification of students needing academic support
- Seamless integration of attendance, marks, and evaluation data
- User-friendly and responsive interface for better accessibility
- Data-driven insights enabling students to take timely corrective actions

2.4 FEASIBILITY STUDY

A feasibility study for the proposed "Eduke: An Academic Performance Prediction System" would assess the viability and potential success of the project from various perspectives, including technical, economic, operational, and legal aspects. Here's an outline of the components typically included in a feasibility study:

2.4.1 TECHNICAL FEASIBILITY

Assessment of the technology required for implementing the Eduke system, including web development tools, databases, AI frameworks, and server configurations. Identification of challenges such as system integration with existing academic platforms, scalability issues, and ensuring the accuracy of AI-driven predictions. Evaluation of infrastructure readiness to support seamless performance and future upgrades.

2.4.2 OPERATIONAL FEASIBILITY

Analysis of how effectively the Eduke system can integrate into the daily operations of educational institutions. Ensuring ease of use for students, parents, class heads, and subject heads to facilitate smooth adoption. Identifying potential challenges such as user training, accessibility issues, and resistance to digital transformation. Assessment of the system’s role in automating attendance tracking, academic performance monitoring, AI-driven predictions, and chatbot interactions to enhance overall efficiency.

2.4.3 ECONOMICAL FEASIBILITY

Cost-benefit analysis to determine the financial viability of the Eduke system. Consideration of initial development costs, ongoing expenses for server maintenance, AI model training, database management, and software updates. Evaluation of potential savings for institutions by reducing manual workload and improving academic tracking efficiency. Comparison of costs against long-term benefits such as enhanced student performance, streamlined administration, and overall institutional growth.

SYSTEM DESIGN

3.1 INTRODUCTION TO SYSTEM DESIGN

System design is the solution to the creation of a new system. This phase is composed of several systems. This phase focuses on the detailed implementation of the feasible system. System design has two phases of development logical and physical design. During logical design phase the analyst describes inputs (sources), out puts (destinations), databases (data sores) and procedures (data flows) all in a format that meats the uses requirements. Design goes through the logical and physical stages of development. At an early stage in designing a new system, the system analyst must have a clear understanding of the objectives, which the design is aiming to fulfil. Second input data and master files (database) have to be designed to meet the requirements of the proposed output. The operational (processing) phases are handled through program construction and testing. The system design includes:

- Output design
- Database design
- Input design
- Form design
- Architectural design
- System modules

3.2 DATABASE DESIGN

Data Base design is the logical form of design of data storage in the form of records in a particular structure in the form of tables with fields which is not transparent to the normal user but it actually acts as the backbone of the system. As we know database is a collection of which helps the system to manage and store data is called database management system. Data base management system builds some form of constraints like integrity constraints, i.e., the primary key or unique key and referential integrity which help to keep data structure storage and access of data from tables efficiently and accurately and take necessary steps to concurrent access of data and avoid redundancy of data in tables by normalization criterions. Normalization is the method of breaking down complex table structures into simple table structures by using certain rules thus reduce redundancy and inconsistency and disk space usage and thus

increase the performance of the system or application which is directly linked to the database design and also solve the problems of anomalies. There are different forms of normalization, some are:

- First normal form (1NF)
- Second normal form (2NF)
- Third normal form (3NF)
- Boyce code normal form
- Forth normal form (4NF)

The database design of the Eduke system follows Boyce-Codd Normal Form (BCNF), ensuring efficient data storage, reducing redundancy, and maintaining consistency. Every non-key attribute is functionally dependent only on the primary key, with no partial or transitive dependencies.

The system is structured using well-defined master tables and transaction tables, each designed to ensure data integrity and scalability.

- First Normal Form (1NF)
  - Each table contains atomic values with no repeating groups or multi-valued attributes.
  - All attributes store single, indivisible pieces of information.
  - The primary keys uniquely identify each record.
  - The database fully satisfies 1NF.

- Second Normal Form (2NF)
  - The database is in 1NF.
  - All non-key attributes are fully dependent on the whole primary key and not just a part of it.
  - Redundant attributes that could cause partial dependencies have been eliminated.
  - The database is in 2NF after ensuring all attributes fully depend on the primary key.

- Third Normal Form (3NF)
  - The database is in 2NF.
  - There are no transitive dependencies (i.e., attributes do not depend on non-primary key attributes).
  - Each entity stores only directly related data, maintaining a clear separation of concerns.
  - The database is in 3NF with no transitive dependencies.

- Boyce-Codd Normal Form (BCNF)
  - The database is in 3NF.
  - Every determinant is a candidate key.
  - Artificial primary keys have been used where necessary for efficient referencing without introducing redundancy.
  - The database is fully normalized up to BCNF, ensuring minimal redundancy and maximum efficiency.

1. Institution

Field	Datatype	Constraint	Description
institution_id	BIGINT	PRIMARY KEY	Unique identifier for the institution
email	VARCHAR(50)	NOT NULL	Email of the institution
institution_name	VARCHAR(50)	NOT NULL	Name of the institution
password	VARCHAR(50)	NOT NULL	Password for authentication

2. Classes

Field	Datatype	Constraint	Description
id	BIGINT	PRIMARY KEY	Unique identifier for the class
class_name	VARCHAR(50)	NOT NULL	Name of the class
class_head	VARCHAR(50)	NOT NULL	Head of the class
email	VARCHAR(50)	NOT NULL	Email of the class head
password	VARCHAR(50)	NOT NULL	Password for authentication
user_id	BIGINT	FOREIGN KEY, NOT NULL	References users(id)
institution_id	BIGINT	FOREIGN KEY, NOT NULL	References institution(institution_id)

3. Subjects

Field	Datatype	Constraint	Description
id	BIGINT	PRIMARY KEY	Unique identifier for the subject
subject_name	VARCHAR(50)	NOT NULL	Name of the subject
subject_head	VARCHAR(50)	NOT NULL	Head of the subject
email	VARCHAR(50)	NOT NULL	Email of the subject head
password	VARCHAR(50)	NOT NULL	Password for authentication
user_id	BIGINT	FOREIGN KEY, NOT NULL	References users(id)
class_id	BIGINT	FOREIGN KEY, NOT NULL	References classes(id)

4. Students

Field	Datatype	Constraint	Description
id	BIGINT	PRIMARY KEY	Unique identifier for the student
roll_no	VARCHAR(50)	NOT NULL	Roll number of the student
password	VARCHAR(50)	NOT NULL	Password for authentication
name	VARCHAR(50)	NOT NULL	Name of the student
email	VARCHAR(50)	NOT NULL	Email of the student
class_id	BIGINT	FOREIGN KEY, NOT NULL	References classes(id)
user_id	BIGINT	FOREIGN KEY, NOT NULL	References users(id)

5. Parents

Field	Datatype	Constraint	Description
id	BIGINT	PRIMARY KEY	Unique identifier for the parent
student_id	VARCHAR(50)	FOREIGN KEY, NOT NULL	References students(id)
password	VARCHAR(50)	NOT NULL	Password for authentication
name	VARCHAR(50)	NULL	Name of the parent
user_id	BIGINT	FOREIGN KEY, NOT NULL	References users(id)

6. Attendance

Field	Datatype	Constraint	Description
id	BIGINT	PRIMARY KEY	Unique identifier for the attendance record
student_id	BIGINT	FOREIGN KEY, NOT NULL	References students(id)
subject_id	BIGINT	FOREIGN KEY, NOT NULL	References subjects(id)
attendance_date	DATE	NOT NULL	Date of attendance record
status	ENUM('present', 'absent')	NOT NULL	Attendance status
created_at	TIMESTAMP	NOT NULL	Time of attendance creation
hour	SMALLINT	NOT NULL	Hour of attendance

7. StudentEvaluations

Field	Datatype	Constraint	Description
id	BIGINT	PRIMARY KEY	Unique identifier for the evaluation
student_id	BIGINT	FOREIGN KEY, NOT NULL	References students(id)
study_time_rating	DOUBLE	NULL	Student's study time rating
sleep_time_rating	DOUBLE	NULL	Student's sleep time rating
class_participation_rating	DOUBLE	NULL	Class participation rating
academic_activity_rating	DOUBLE	NULL	Academic activity rating
attendance_percentage	DOUBLE	NULL	Attendance percentge
mark_percentage	DOUBLE	NULL	Mark percentage
subject_id	BIGINT	NULL	References subjects(id)

8. Users

Field	Datatype	Constraint	Description
id	BIGINT	PRIMARY KEY	Unique identifier for the user
role	ENUM('class_head', 'subject_head', 'student', 'parent')	NOT NULL	Role of the user

9. StudyMaterials

Field	Datatype	Constraint	Description
id	BIGINT	PRIMARY KEY	Unique identifier for the study material
file_url	LONGTEXT	NULL	Link to study material
created_at	DATE	NOT NULL	Creation time of study material
class_id	BIGINT	FOREIGN KEY, NOT NULL	Reference to the class
subject_id	BIGINT	FOREIGN KEY, NOT NULL	Reference to the subject
announcement	LONGTEXT	NULL	Content of the announcement

10. Chat

Field	Datatype	Constraint	Description
id	INT	PRIMARY KEY	Unique identifier for the chat message
message	TEXT	NOT NULL	Chat message content
sender_id	INT	FOREIGN KEY, NOT NULL	Reference to the sender
receiver_id	INT	FOREIGN KEY, NOT NULL	Reference to the receiver
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Time of message creation

11. Quizzes

Field	Datatype	Constraint	Description
id	INT	PRIMARY KEY	Unique identifier for the quiz
name	VARCHAR(255)	NOT NULL	Name of the quiz
class_id	INT	FOREIGN KEY, NOT NULL	Reference to the class
subject_id	INT	FOREIGN KEY, NOT NULL	Reference to the subject

12. QuizQuestions

Field	Datatype	Constraint	Description
id	INT	PRIMARY KEY	Unique identifier for the quiz question
question	TEXT	NOT NULL	Quiz question text
option_a	TEXT	NOT NULL	Option A
option_b	TEXT	NOT NULL	Option B
option_c	TEXT	NOT NULL	Option C
option_d	TEXT	NOT NULL	Option D
correct_option	ENUM('A', 'B', 'C', 'D')	NOT NULL	Correct answer option
quiz_id	INT	FOREIGN KEY, NOT NULL	Reference to the quiz

13. QuizResponses

Field	Datatype	Constraint	Description
id	INT	PRIMARY KEY	Unique identifier for the quiz response
student_response	ENUM('A', 'B', 'C', 'D')	NOT NULL	Student's selected answer
question_id	INT	FOREIGN KEY, NOT NULL	Reference to the quiz question
student_id	INT	FOREIGN KEY, NOT NULL	Reference to the student

14. Announcements

Field	Datatype	Constraint	Description
id	INT	PRIMARY KEY	Unique identifier for the announcement
class_id	INT	FOREIGN KEY, NOT NULL	Reference to the class
message	LONGTEXT	NOT NULL	Content of the announcement
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Time of announcement creation

3.3 ENTITY-RELATIONSHIP MODEL

An entity relationship diagram (ERD), also known as an entity relationship model, is a graphical representation that depicts relationships among people, objects, places, concepts or events within an information technology (IT) system. An ERD uses data modeling techniques that can help define business processes and serve as the foundation for a relational database.

Entity relationship diagrams provide a visual starting point for database design that can also be used to help determine information system requirements throughout an organization. After a relational database is rolled out, an ERD can still serve as a reference point, should any debugging or business process re-engineering be needed later.

However, while an ERD can be useful for organizing data that can be represented by a relational structure, it can't sufficiently represent semi-structured or unstructured data. It's also unlikely to be helpful on its own in integrating data into a pre-existing information system.

ERDs are generally depicted in one or more of the following models:

- A conceptual data model, which lacks specific detail but provides an overview of the scope of the project and how data sets relate to one another.
- A logical data model, which is more detailed than a conceptual data model, illustrating specific attributes and relationships among data points. While a conceptual data model does not need to be designed before a logical data

model, a physical data model is based on a logical data model.

- A physical data model, which provides the blueprint for a physical manifestation such as a relational database -- of the logical data model. One or more physical data models can be developed based on a logical data model.

3.3.1 DATA DICTIONARY

A Data Dictionary is a collection of names, definitions, and attributes about data elements that are being used or captured in a database, information system, or part of a research project. It describes the meanings and purposes of data elements within the context of a project, and provides guidance on interpretation, accepted meanings and representation. A Data Dictionary also provides metadata about data elements. The metadata included in a Data Dictionary can assist in defining the scope and characteristics of data elements, as well the rules for their usage and application. Data Dictionaries are useful for a number of reasons. In short, they:

- Assist in avoiding data inconsistencies across a project.
- Help define conventions that are to be used across a project.
- Provide consistency in the collection and use of data across multiple members of a research team.
- Make data easier to analyst.
- Enforce the use of Data Standards.

Data Standards are rules that govern the way data are collected, recorded, and represented. Standards provide a commonly understood reference for the interpretation and use of data sets.

By using standards, researchers in the same disciplines will know that the way their data are being collected and described will be the same across different projects. Using Data Standards as part of a well-crafted Data Dictionary can help increase the usability of your research data, and will ensure that data will be recognizable and usable beyond the immediate research team.

3.4 OBJECT-ORIENTED DESIGN – UML DIAGRAMS

UML stands for Unified Modelling Language. UML is a language for specifying, visualizing and documenting the system. This is the step while developing any product after analysis. The goal from this is to produce a model of the entities involved in the project which later need to be built. The representation of the entities that are to be used in the product being developed need to be designed. Software design is a process that gradually changes as various new, better and more complete methods with a broader understanding of the of the whole problem in general come into existence. There are various kinds of methods in software design. They are as follows:

- Use case diagram
- Activity diagram
- Sequence diagram
- Class diagram

Use case Diagrams:

Use case diagrams model behaviour within a system and helps the developers understand of what the user require. The stick man represents what's called an actor. An actor represents an outside entity - either human or technological. Use case diagrams can be useful for getting an overall view of the system and clarifying who can do and more importantly what they can't do. Use case Diagram consists of use cases and actors and shows the interaction between the use case and actors. The purpose is to show the interactions between use cases and actor. To represent the system requirements from user's perspective. It must be remembered that the use-cases are the functions that are to be performed in the module. An actor could be the end-user of the system or an external system. In this project mainly we have five users:

- Admin
- Class Head
- Subject head
- Student
- Parent

ADMIN

The admin in the Eduke system is responsible for managing and overseeing the overall functioning of the platform. They begin by registering an account and logging into the system to access the admin dashboard. The admin has the authority to create, edit, or delete classes by adding class names, assigning class heads, and managing class email details. Similarly, they can manage subjects by creating new subjects, assigning subject heads, linking them to classes, and modifying or deleting subject details when necessary. Student management is another key responsibility, where the admin can add new students by entering their name, roll number, email, and class assignment, as well as edit or delete student records when required. Additionally, the admin can update institution details, such as modifying the institution name and changing passwords for security. Once their tasks are completed, they can securely log out of the system. Through these responsibilities, the admin plays a crucial role in maintaining the structure, organization, and smooth operation of the Eduke system.

CLASS HEAD

The class head in the Eduke system is responsible for managing and overseeing the activities of a specific class. They can log into the system and access the class head dashboard, where they can view class details, including the list of students and subjects under their supervision. The class head can also manage announcements by viewing or adding important updates for students. Additionally, they have access to student profiles and academic performance reports, allowing them to monitor progress and view score predictions to assess students' academic standing. Communication is another key responsibility, as they can send messages to students and parents to provide guidance and updates. Furthermore, the class head has the ability to edit their details, such as updating their name and password. Finally, they can securely log out of the system after completing their tasks. Through these responsibilities, the class head plays a crucial role in ensuring smooth class management and academic monitoring within the Eduke system.

SUBJECT HEAD

The subject head in the Eduke system is responsible for managing subject-related activities and overseeing student performance. After logging in, they gain access to

the subject head dashboard, where they can view details about their assigned subject and the associated class. One of their key responsibilities is monitoring student profiles and performance, including viewing individual student records, printing reports, and checking score predictions. The subject head also manages student attendance by adding, editing, deleting, and viewing attendance records. Similarly, they handle student marks by recording, updating, and reviewing student scores. Additionally, they assess student evaluations by adding, modifying, or reviewing evaluation details. Another crucial role of the subject head is creating quizzes by entering quiz names, adding questions with multiple-choice options, selecting correct answers, and submitting the quizzes for students. They can also upload announcements and study materials to assist students in their learning process. Communication is another aspect of their role, as they can send messages to students and parents regarding academic updates or important information. Furthermore, they have the ability to edit their profile details, such as updating their name and password. Once their tasks are completed, they can securely log out of the Eduke system. Through these responsibilities, the subject head ensures effective subject management, student assessment, and academic guidance within the system.

STUDENT

The student role in the Eduke system is designed to provide a seamless academic experience by allowing students to access essential information, interact with their subjects, and track their progress. Upon logging in, students can view their personalized dashboard, which grants access to various academic resources. They can check class and subject details, view important announcements, and download study materials shared by subject heads. Students also have the ability to participate in quizzes by selecting quizzes, answering questions, and instantly viewing their scores.

To monitor their academic performance, students can access their profile, where they can review their marks, attendance, and evaluations. Additionally, the system provides an AI-driven score prediction feature, offering insights into expected academic performance based on historical data. Eduke AI, an interactive chatbot, assists students by answering academic queries, providing study tips, and offering guidance in a friendly and supportive manner. Students can also communicate with

their teachers by sending messages for academic assistance or clarifications. Furthermore, they can edit their profile information, including updating their name and password, ensuring account security. Once they have completed their tasks, they can securely log out of the system. Through these features, the Eduke system empowers students with valuable tools to enhance their learning experience and academic performance.

PARENT

The parent role in the Eduke system is designed to help parents stay informed about their child's academic progress and overall performance. Upon logging in, parents can access their personalized dashboard, which provides an overview of essential academic details. They can view class and subject information, allowing them to stay updated on what their child is studying. One of the key features available to parents is the ability to monitor student evaluations. They can add, edit, or review their child's evaluations, helping them assess study habits and engagement. Additionally, parents can view their child's profile, academic performance, and even print a report for reference. The system also includes a score prediction feature that analyzes marks, attendance, and evaluations to provide an estimated academic outcome. Eduke AI, the integrated chatbot, assists parents by answering queries related to student performance, academic guidance, and system-related concerns. Parents can also communicate with subject heads by sending messages to clarify doubts or discuss academic concerns. Furthermore, they have the option to edit their profile details, ensuring their information is up-to-date. Once their tasks are completed, parents can securely log out of the system. With these features, the Eduke system empowers parents to actively engage in their child's education, fostering a supportive learning environment.

Activity Diagram:

The purpose is to show the activities which the users performed. Actives are shown parallel and sequentially in which order they are performed. Some activities are joined and split according to its flow. Flow of data is represented using arrows.

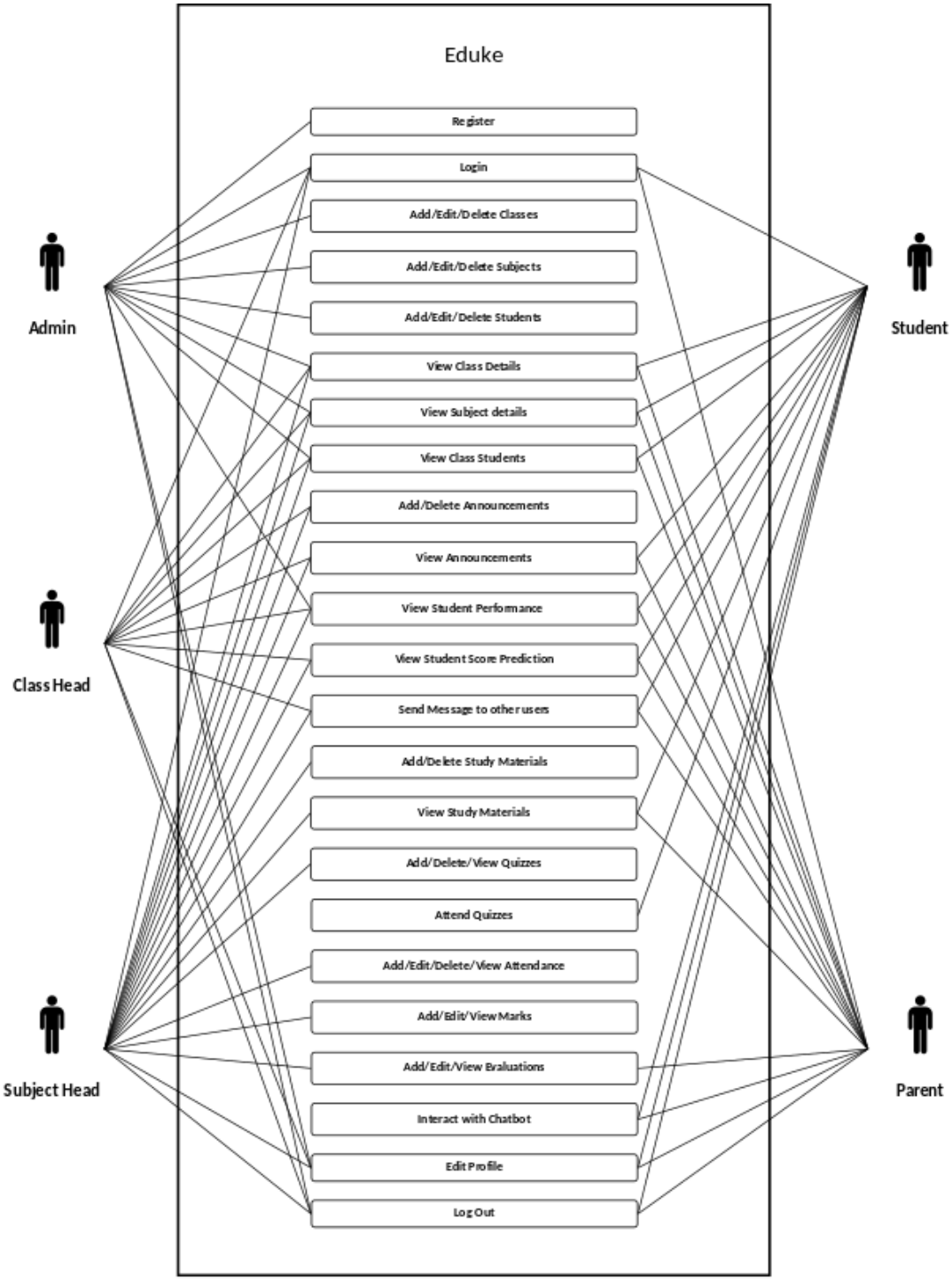
Sequence Diagram:

The purpose is to show the sequential flow through of activities. In other Words, we call it mapping processes in terms of data transfers from the actor through corresponding objects. To represent the logical flow of data with respect to a process. It must be remembered that the sequence diagram display objects and not the classes.

Class Diagram:

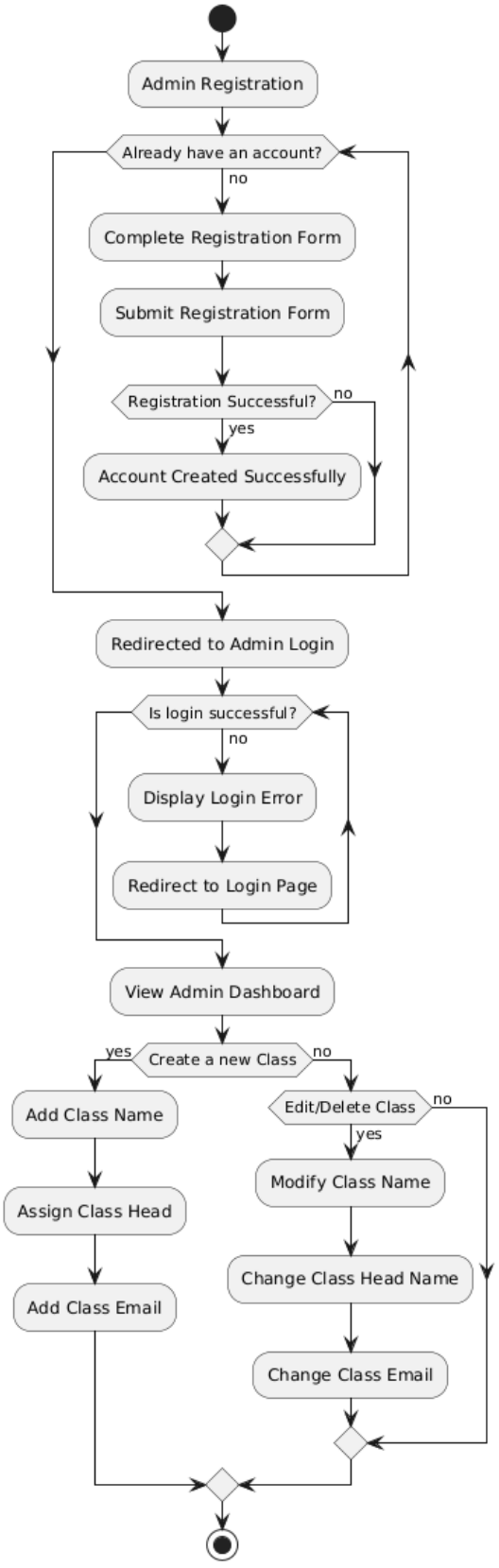
This is one of the most important of the diagrams in development. The diagram breaks the class into three layers. One has the name, the second describes its attributes and the third its methods. The private attributes are represented by a padlock to left of the name. The relationships are drawn between the classes. Developers use the Class Diagram to develop the classes. Analyses use it to show the details of the system. Architects look at class diagrams to see if any class has too many functions and see if they are required to be split.

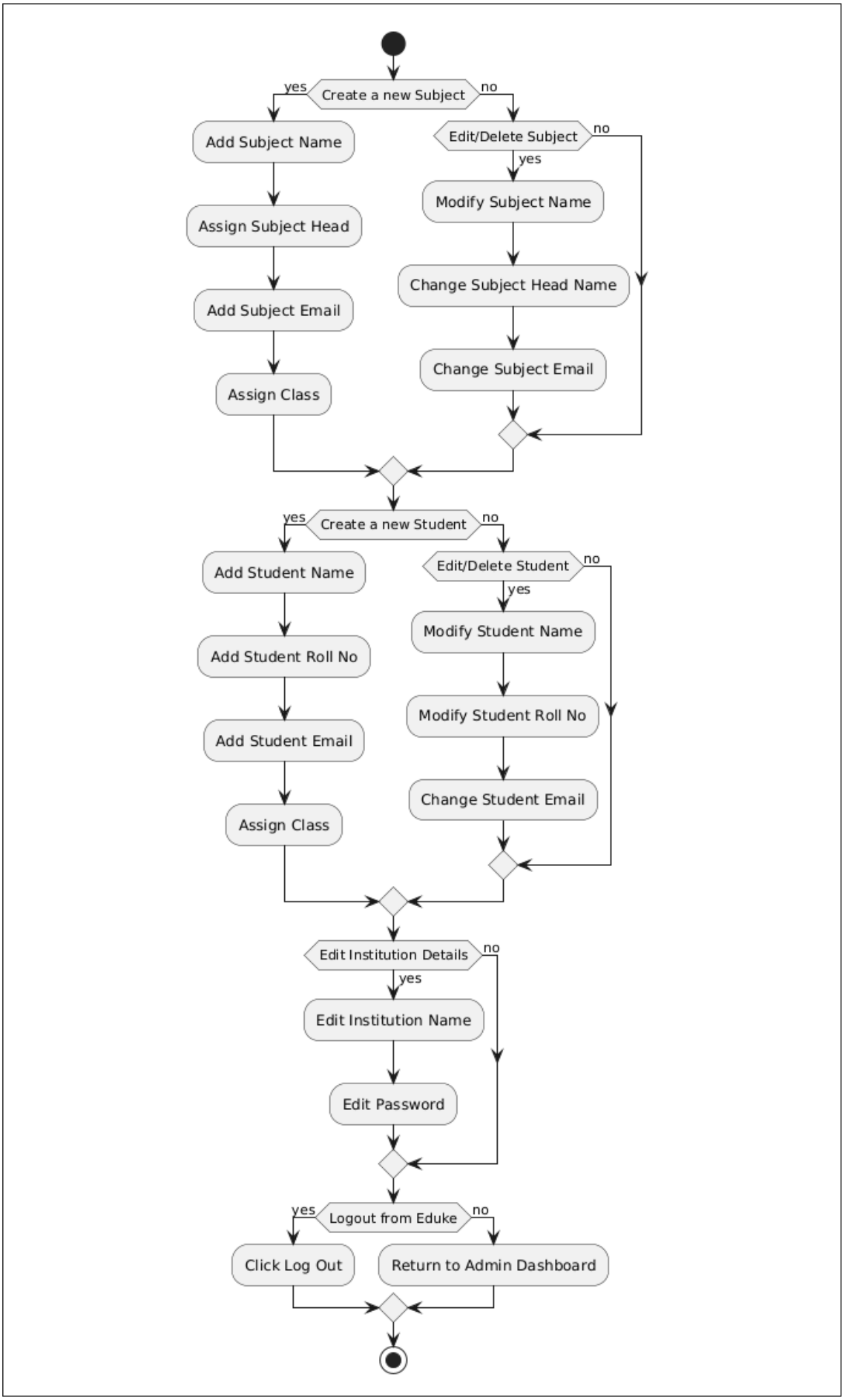
3.4.1 USECASE DIAGRAM



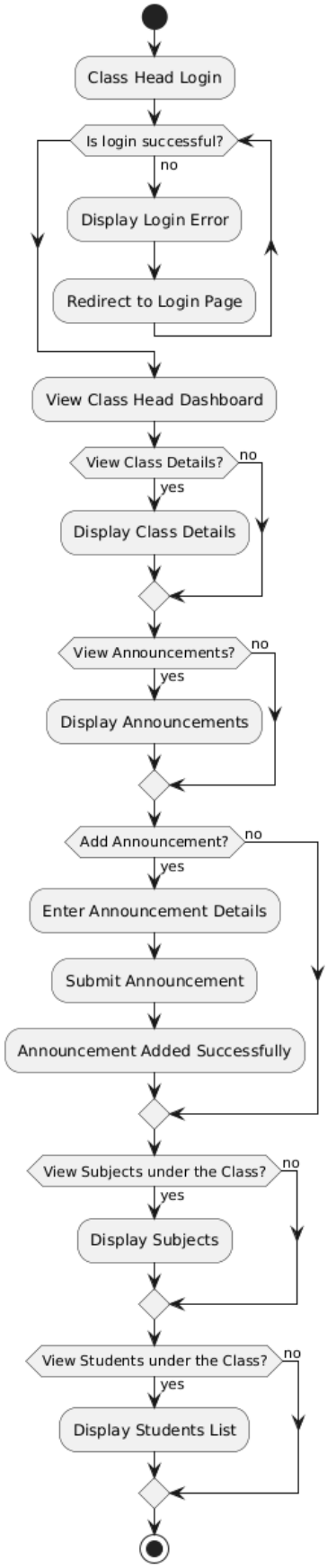
3.4.2 ACTIVITY DIAGRAM

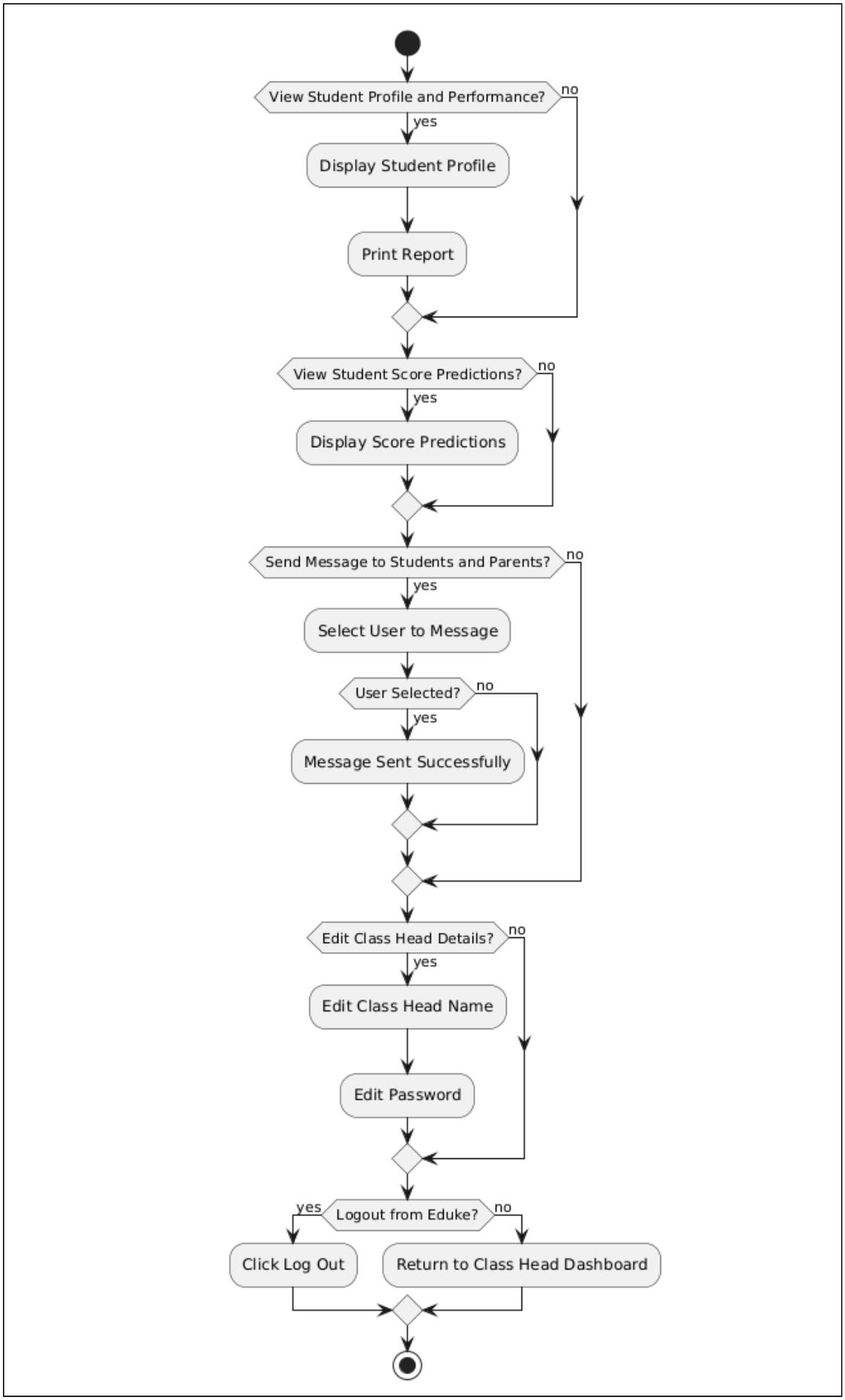
Admin



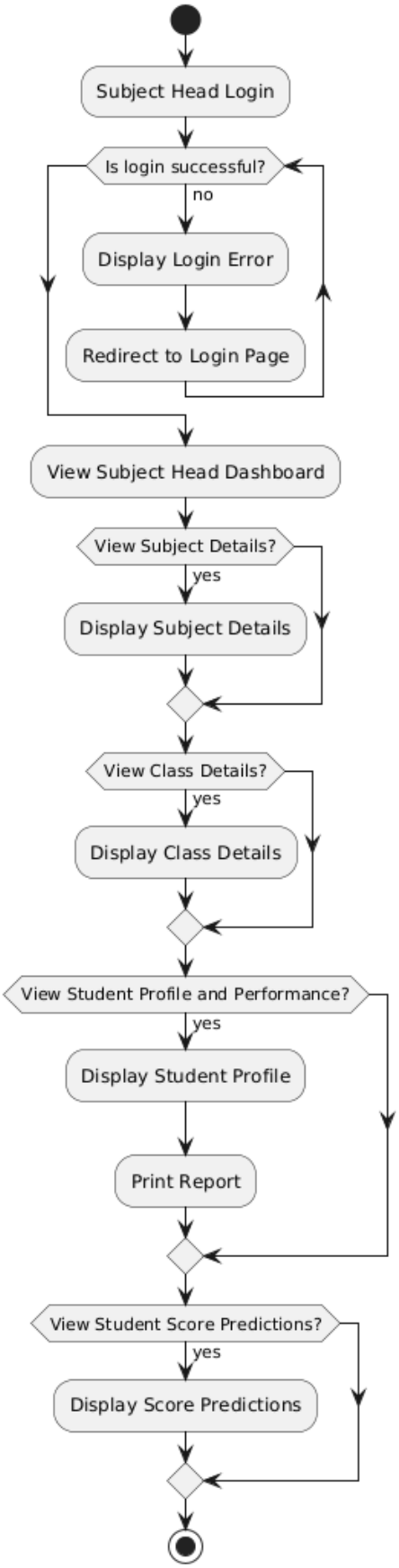

Create a new Subject
yes
no
Add Subject Name
Assign Subject Head
Add Subject Email
Assign Class
Edit/Delete Subject
no
yes
Modify Subject Name
Change Subject Head Name
Change Subject Email
Create a new Student
yes
no
Add Student Name
Add Student Roll No
Add Student Email
Assign Class
Edit/Delete Student
no
yes
Modify Student Name
Modify Student Roll No
Change Student Email
Edit Institution Details
no
yes
Edit Institution Name
Edit Password
Logout from Eduke
yes
no
Click Log Out
Return to Admin Dashboard

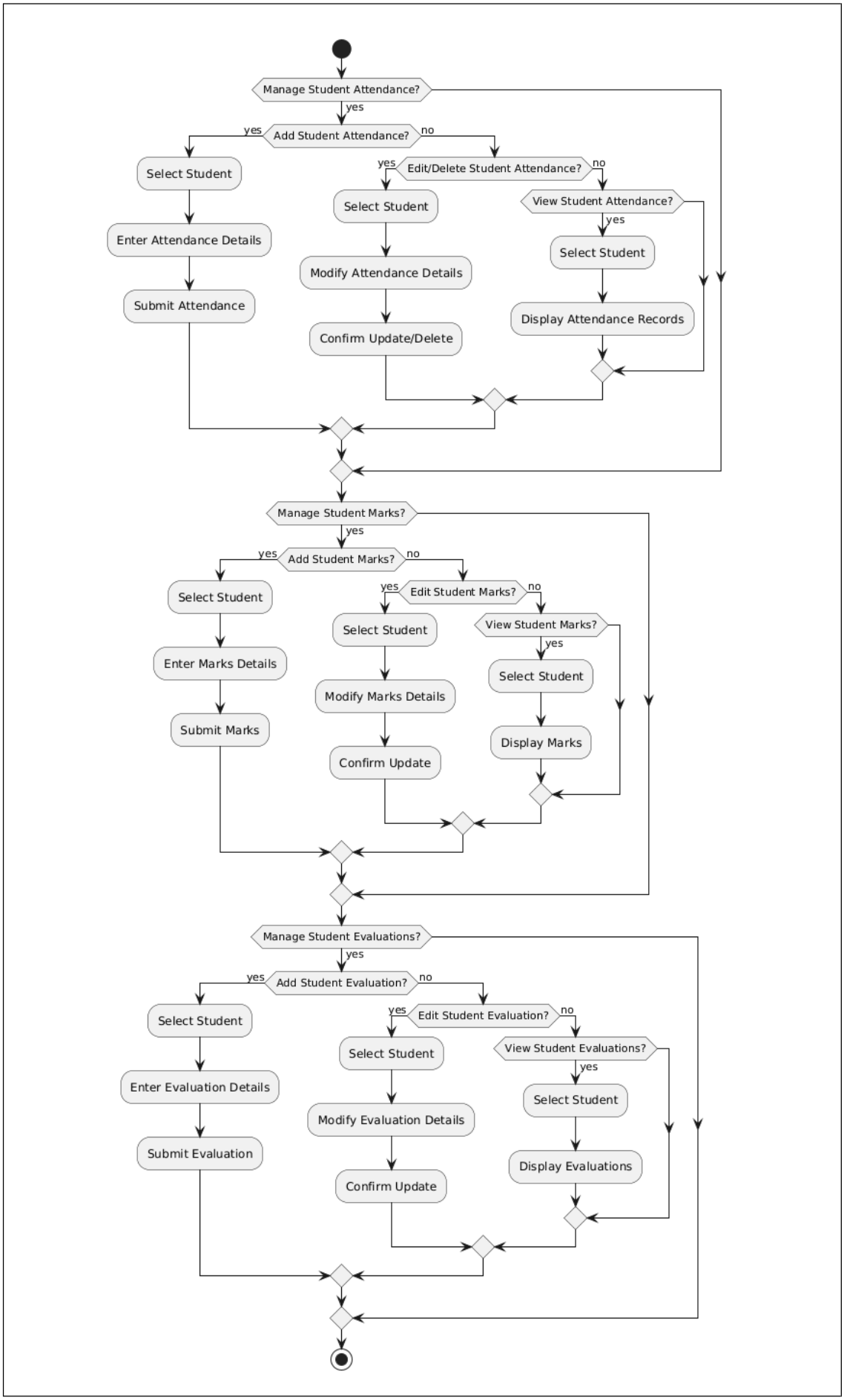
Class Head

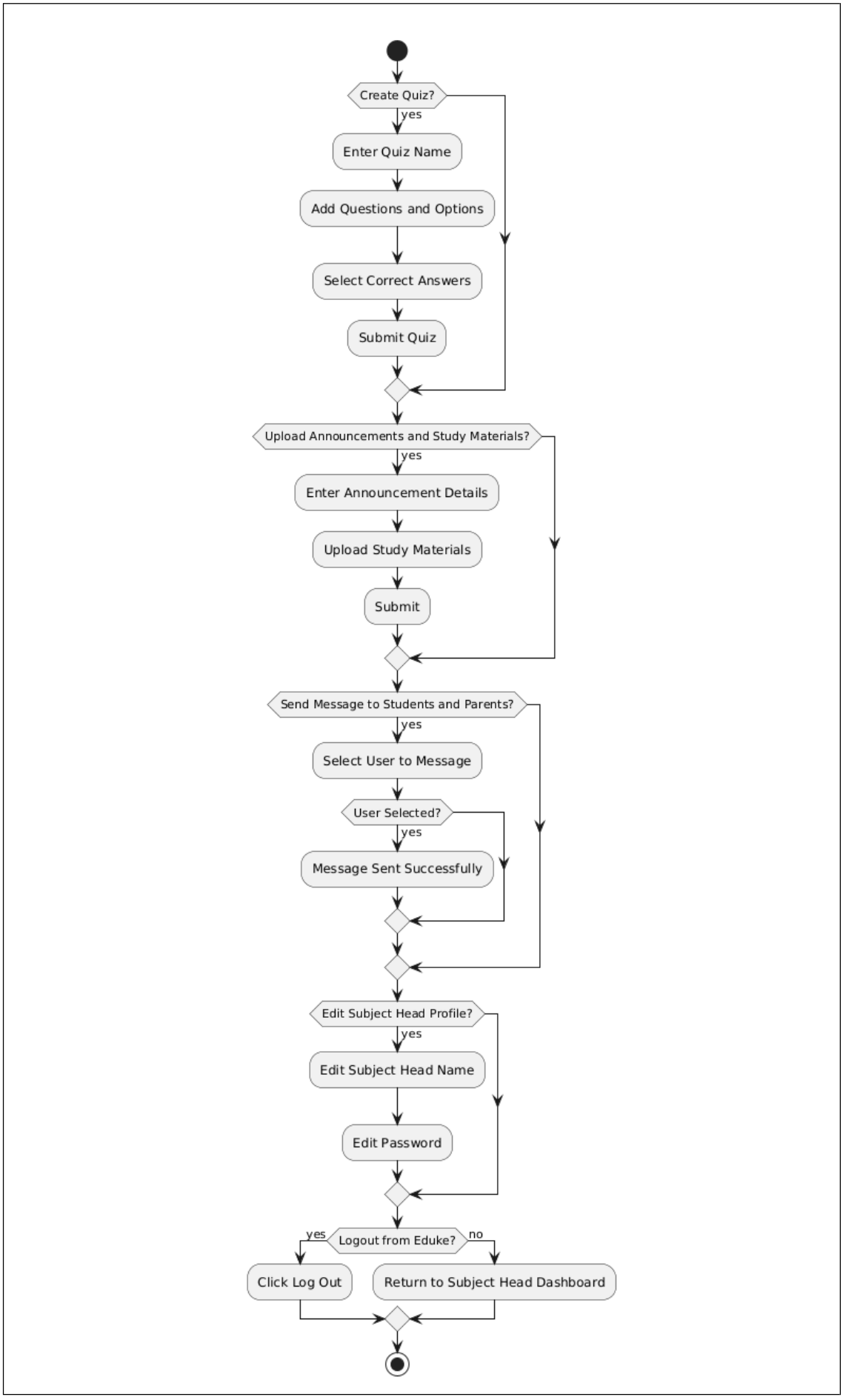

Class Head Login
Is login successful?
no
Display Login Error
Redirect to Login Page
View Class Head Dashboard
View Class Details?
no
yes
Display Class Details
View Announcements?
no
yes
Display Announcements
Add Announcement?
no
yes
Enter Announcement Details
Submit Announcement
Announcement Added Successfully
View Subjects under the Class?
no
yes
Display Subjects
View Students under the Class?
no
yes
Display Students List


View Student Profile and Performance?
no
yes
Display Student Profile
Print Report
View Student Score Predictions?
no
yes
Display Score Predictions
Send Message to Students and Parents?
no
yes
Select User to Message
User Selected?
no
yes
Message Sent Successfully
Edit Class Head Details?
no
yes
Edit Class Head Name
Edit Password
yes
Logout from Eduke?
no
Click Log Out
Return to Class Head Dashboard

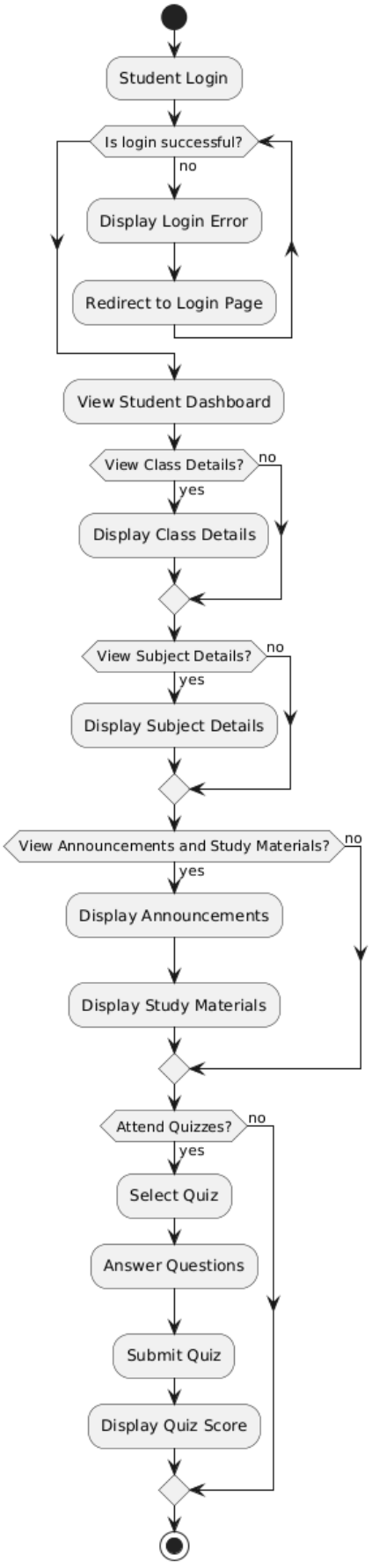
Subject Head

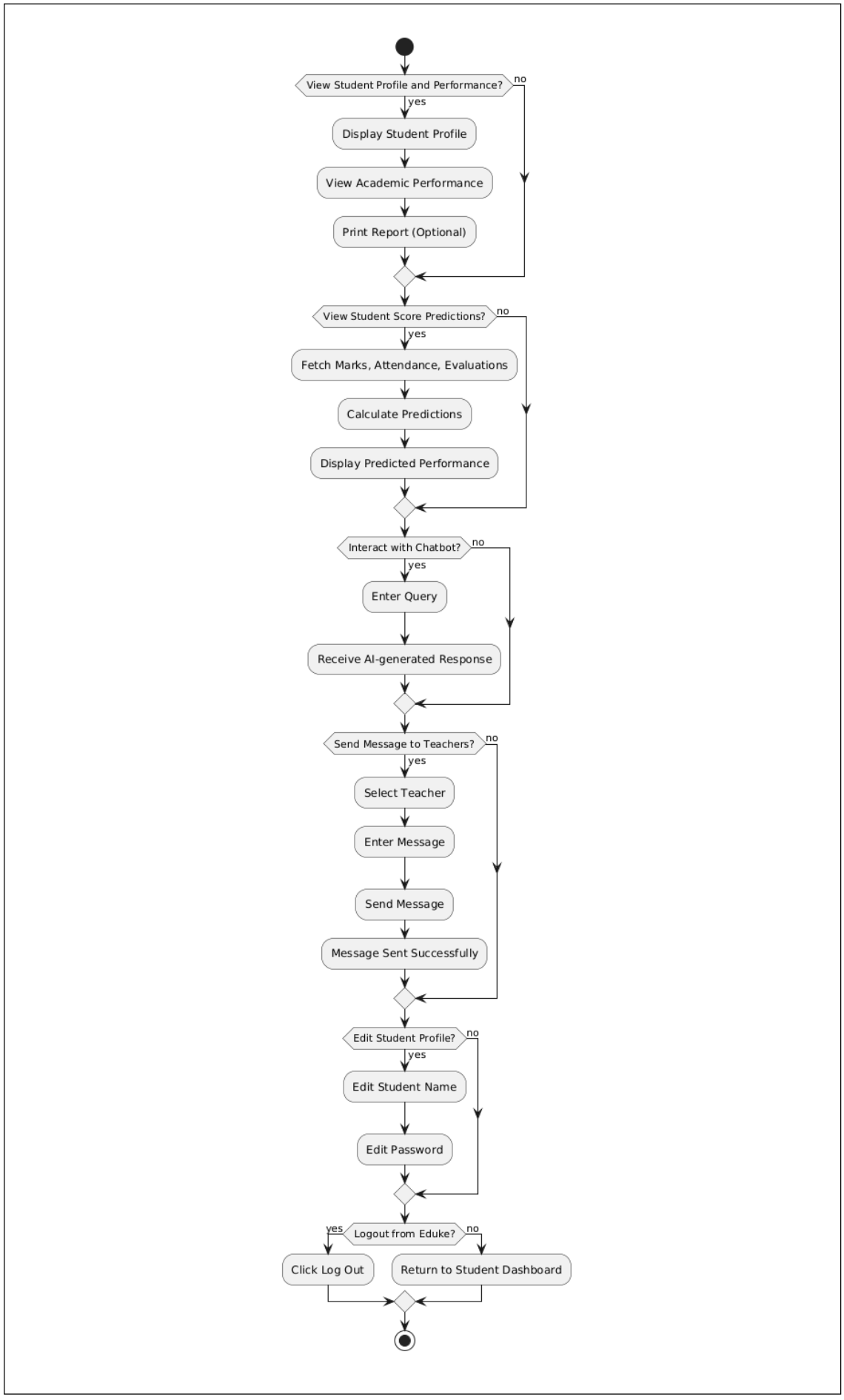



Manage Student Attendance?
yes
Add Student Attendance?
yes
no
Select Student
Enter Attendance Details
Submit Attendance
Edit/Delete Student Attendance?
yes
no
Select Student
Modify Attendance Details
Confirm Update/Delete
View Student Attendance?
yes
Select Student
Display Attendance Records
Manage Student Marks?
yes
Add Student Marks?
yes
no
Select Student
Enter Marks Details
Submit Marks
Edit Student Marks?
yes
no
Select Student
Modify Marks Details
Confirm Update
View Student Marks?
yes
Select Student
Display Marks
Manage Student Evaluations?
yes
Add Student Evaluation?
yes
no
Select Student
Enter Evaluation Details
Submit Evaluation
Edit Student Evaluation?
yes
no
Select Student
Modify Evaluation Details
Confirm Update
View Student Evaluations?
yes
Select Student
Display Evaluations

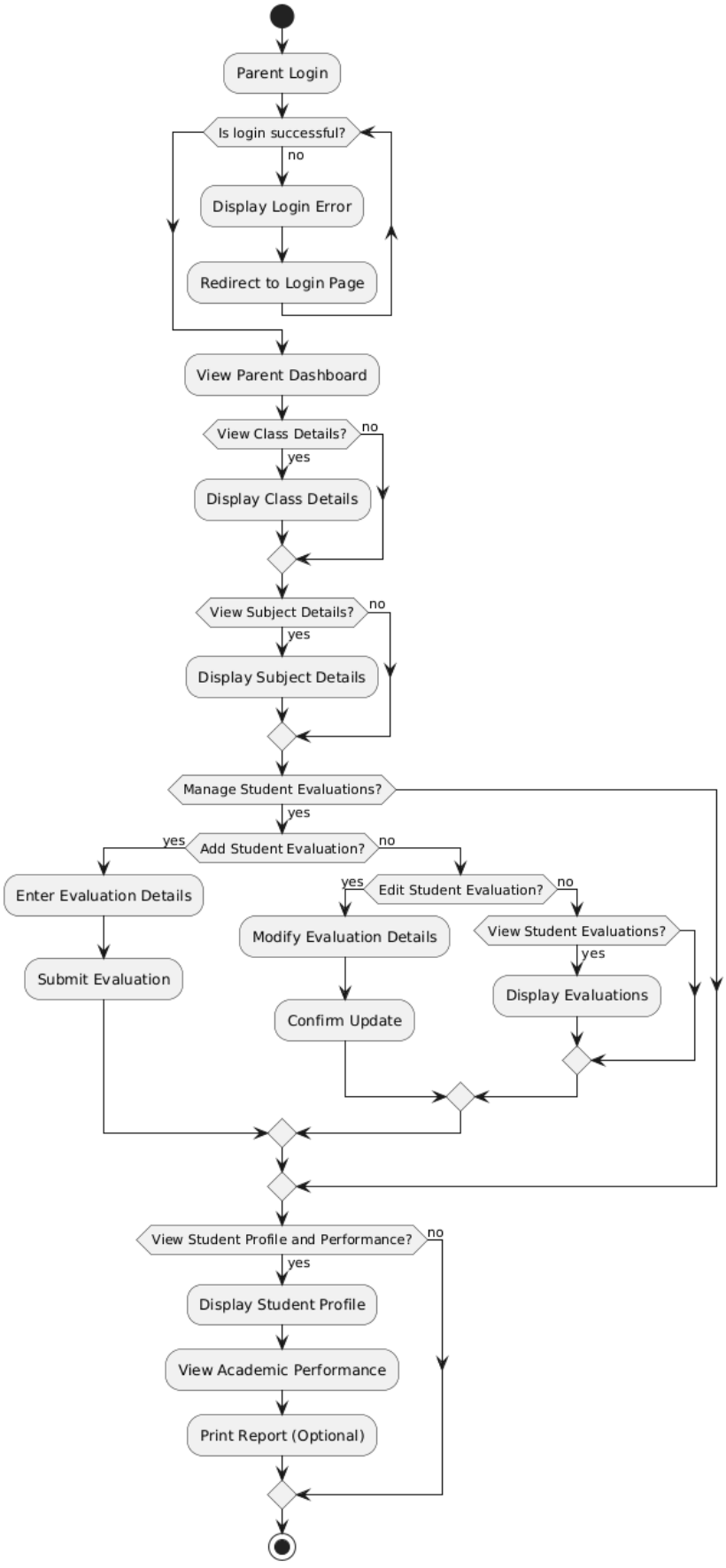

Create Quiz?
yes
Enter Quiz Name
Add Questions and Options
Select Correct Answers
Submit Quiz
Upload Announcements and Study Materials?
yes
Enter Announcement Details
Upload Study Materials
Submit
Send Message to Students and Parents?
yes
Select User to Message
User Selected?
yes
Message Sent Successfully
Edit Subject Head Profile?
yes
Edit Subject Head Name
Edit Password
yes
Logout from Eduke?
no
Click Log Out
Return to Subject Head Dashboard

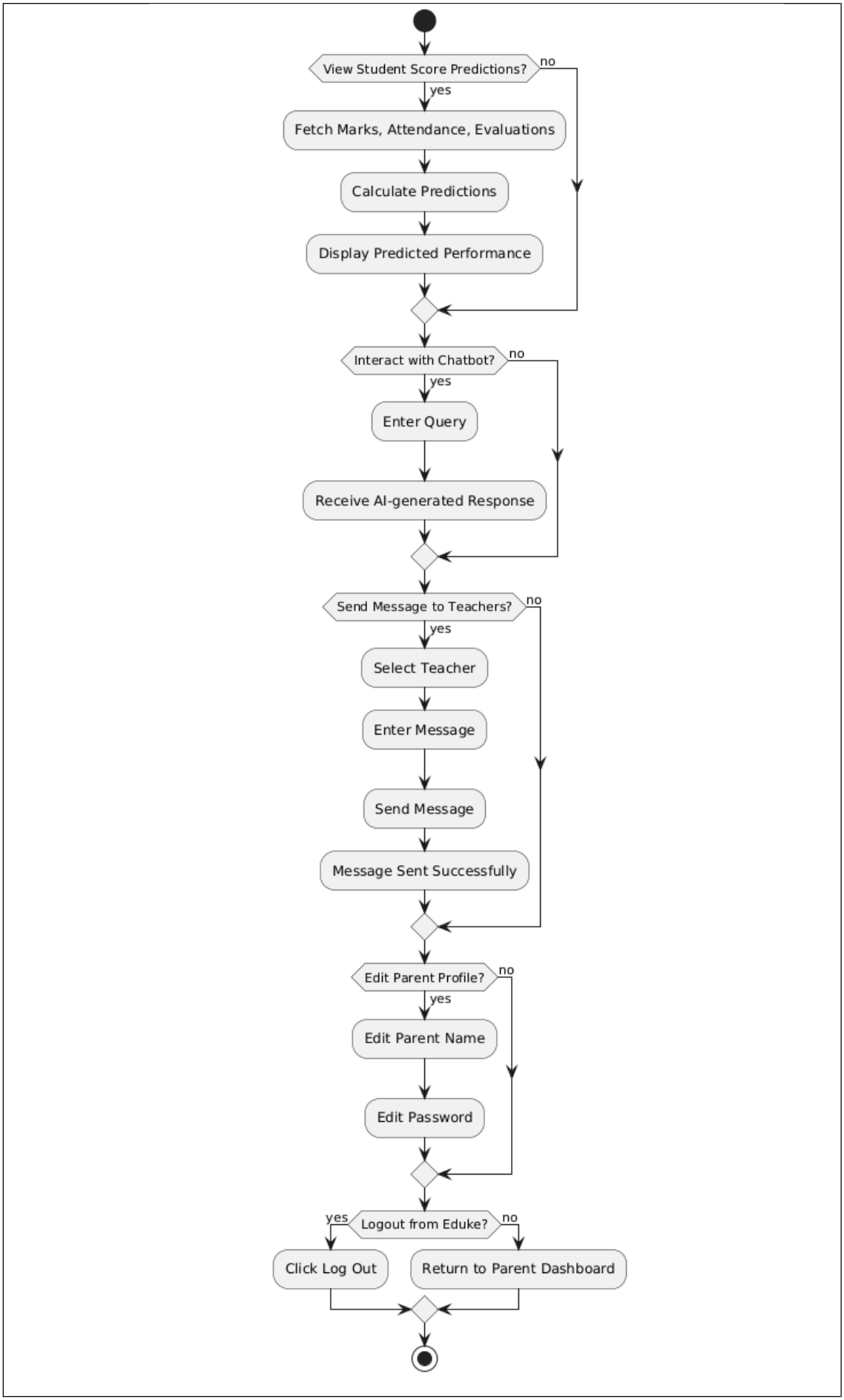
Student


Student Login
Is login successful?
no
Display Login Error
Redirect to Login Page
View Student Dashboard
View Class Details?
no
yes
Display Class Details
View Subject Details?
no
yes
Display Subject Details
View Announcements and Study Materials?
no
yes
Display Announcements
Display Study Materials
Attend Quizzes?
no
yes
Select Quiz
Answer Questions
Submit Quiz
Display Quiz Score

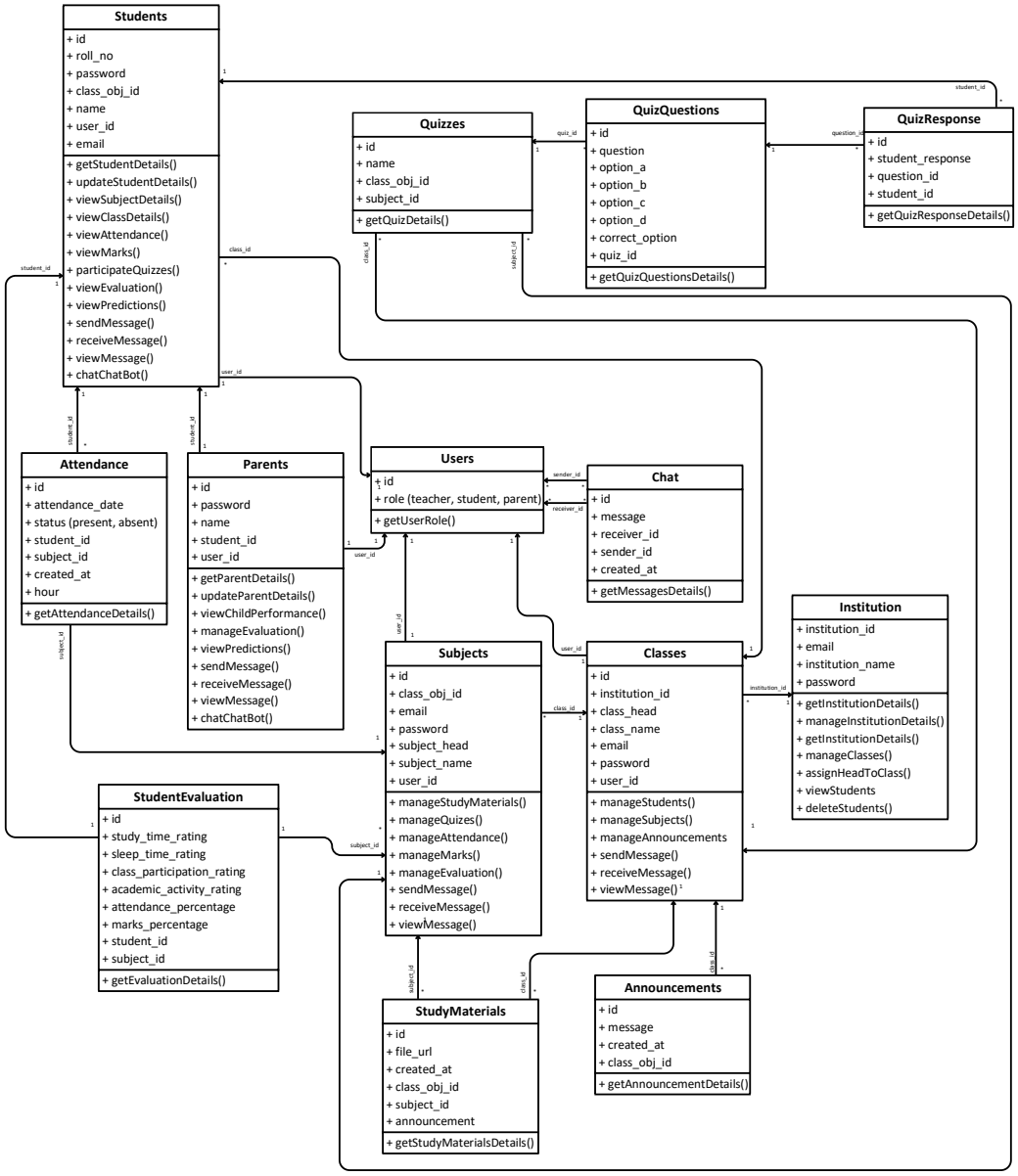

View Student Profile and Performance?
no
yes
Display Student Profile
View Academic Performance
Print Report (Optional)
View Student Score Predictions?
no
yes
Fetch Marks, Attendance, Evaluations
Calculate Predictions
Display Predicted Performance
Interact with Chatbot?
no
yes
Enter Query
Receive AI-generated Response
Send Message to Teachers?
no
yes
Select Teacher
Enter Message
Send Message
Message Sent Successfully
Edit Student Profile?
no
yes
Edit Student Name
Edit Password
yes
Logout from Eduke?
no
Click Log Out
Return to Student Dashboard

Parent


Parent Login
Is login successful?
no
Display Login Error
Redirect to Login Page
View Parent Dashboard
View Class Details?
no
yes
Display Class Details
View Subject Details?
no
yes
Display Subject Details
Manage Student Evaluations?
yes
Add Student Evaluation?
yes
no
Enter Evaluation Details
Submit Evaluation
Edit Student Evaluation?
yes
no
Modify Evaluation Details
Confirm Update
View Student Evaluations?
yes
Display Evaluations
View Student Profile and Performance?
no
yes
Display Student Profile
View Academic Performance
Print Report (Optional)

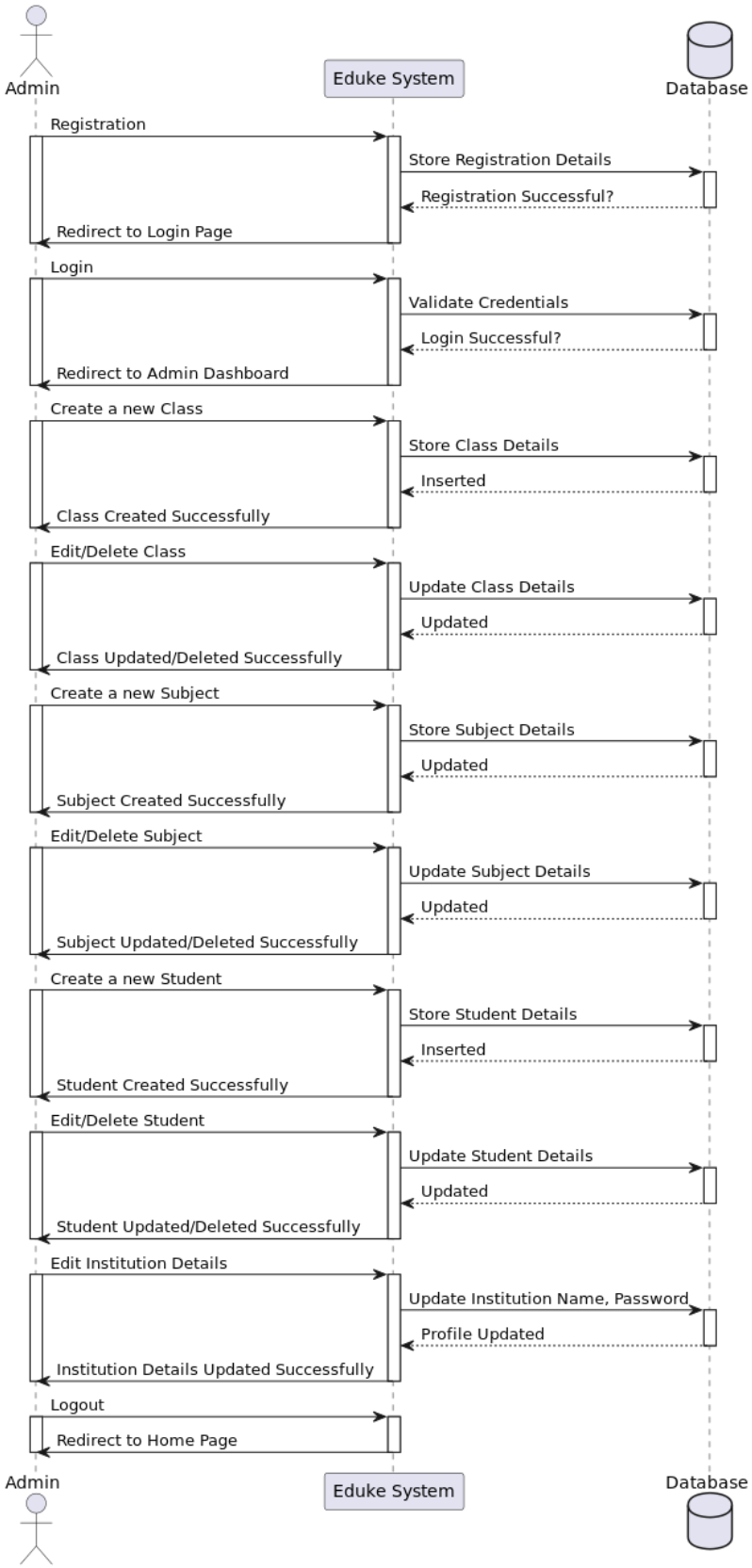

View Student Score Predictions?
no
yes
Fetch Marks, Attendance, Evaluations
Calculate Predictions
Display Predicted Performance
Interact with Chatbot?
no
yes
Enter Query
Receive AI-generated Response
Send Message to Teachers?
no
yes
Select Teacher
Enter Message
Send Message
Message Sent Successfully
Edit Parent Profile?
no
yes
Edit Parent Name
Edit Password
yes
Logout from Eduke?
no
Click Log Out
Return to Parent Dashboard

3.4.3 CLASS DIAGRAM

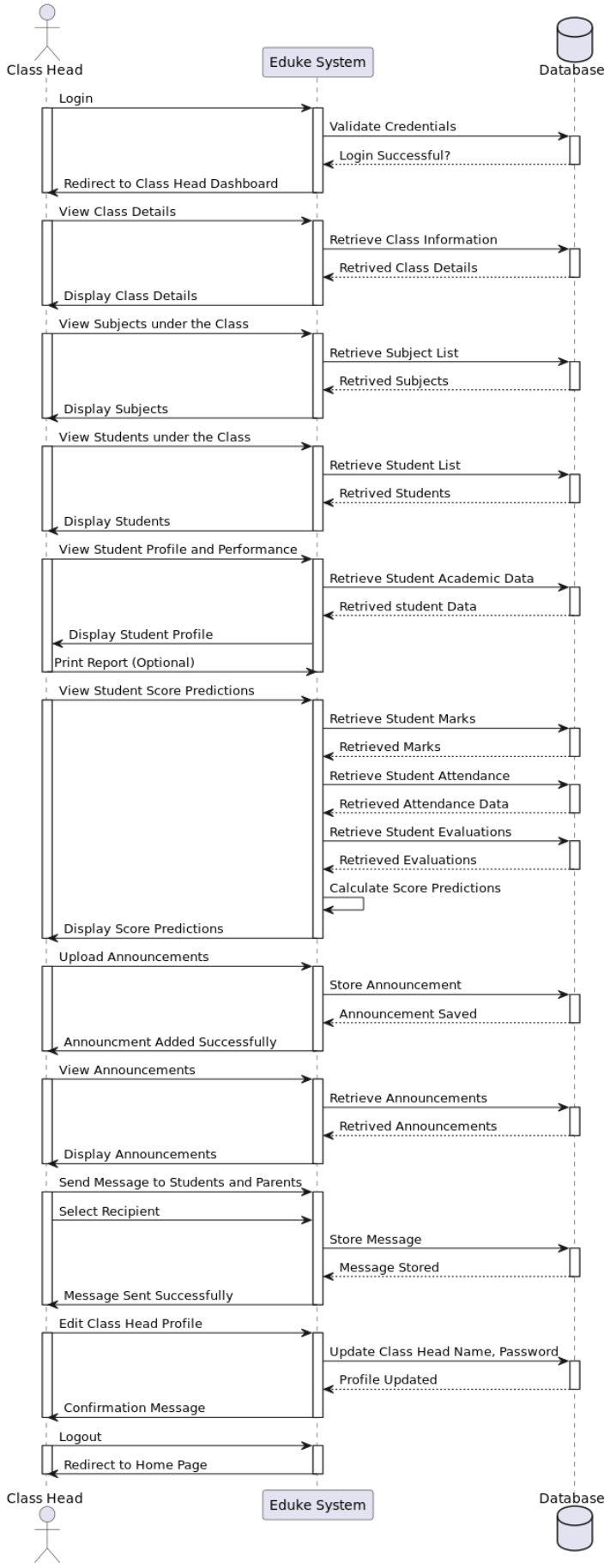


3.4.4 SEQUENCE DIAGRAM

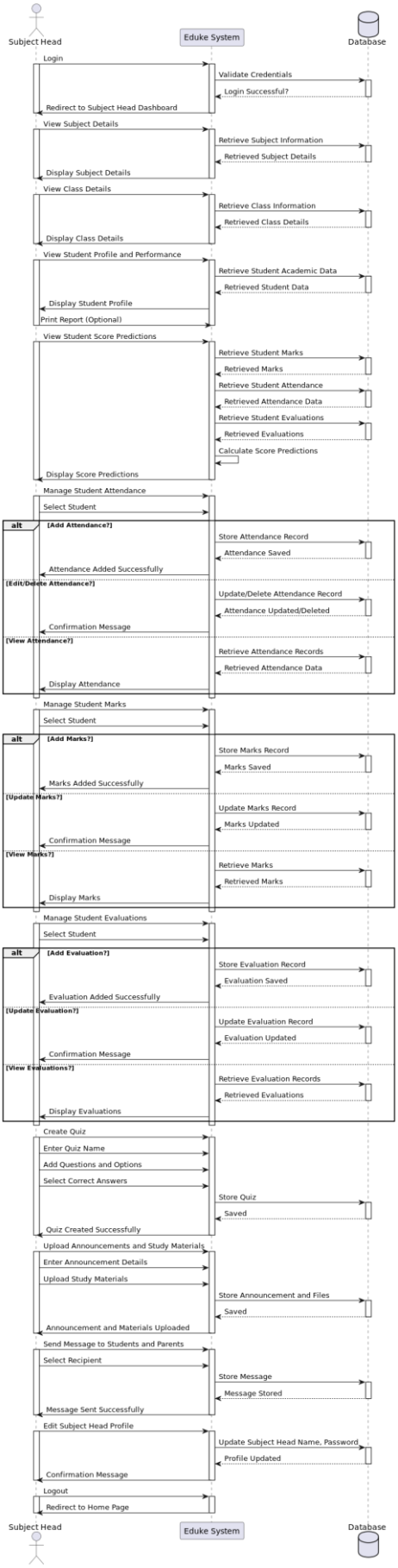
Admin



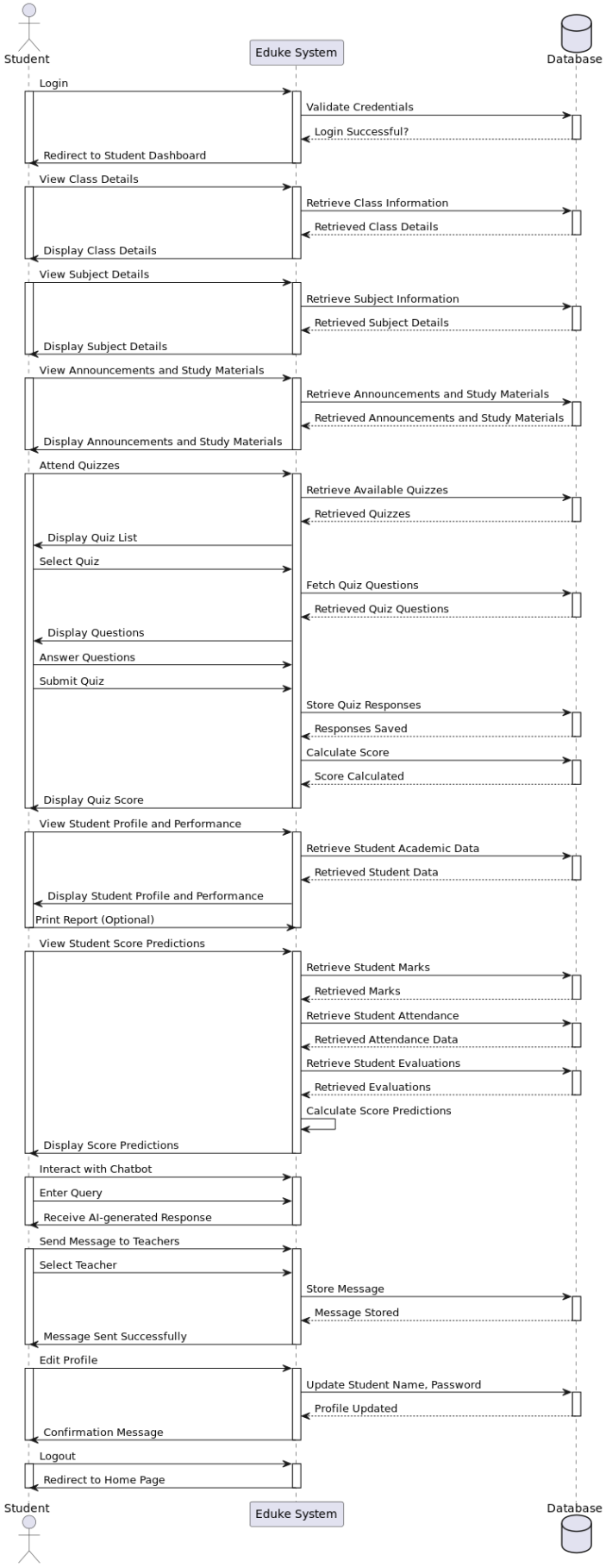
Class Head


Class Head
Eduke System
Database
Login
Validate Credentials
Login Successful?
Redirect to Class Head Dashboard
View Class Details
Retrieve Class Information
Retrived Class Details
Display Class Details
View Subjects under the Class
Retrieve Subject List
Retrived Subjects
Display Subjects
View Students under the Class
Retrieve Student List
Retrived Students
Display Students
View Student Profile and Performance
Retrieve Student Academic Data
Retrived student Data
Display Student Profile
Print Report (Optional)
View Student Score Predictions
Retrieve Student Marks
Retrieved Marks
Retrieve Student Attendance
Retrieved Attendance Data
Retrieve Student Evaluations
Retrieved Evaluations
Calculate Score Predictions
Display Score Predictions
Upload Announcements
Store Announcement
Announcement Saved
Announcment Added Successfully
View Announcements
Retrieve Announcements
Retrived Announcements
Display Announcements
Send Message to Students and Parents
Select Recipient
Store Message
Message Stored
Message Sent Successfully
Edit Class Head Profile
Update Class Head Name, Password
Profile Updated
Confirmation Message
Logout
Redirect to Home Page
Class Head
Eduke System
Database

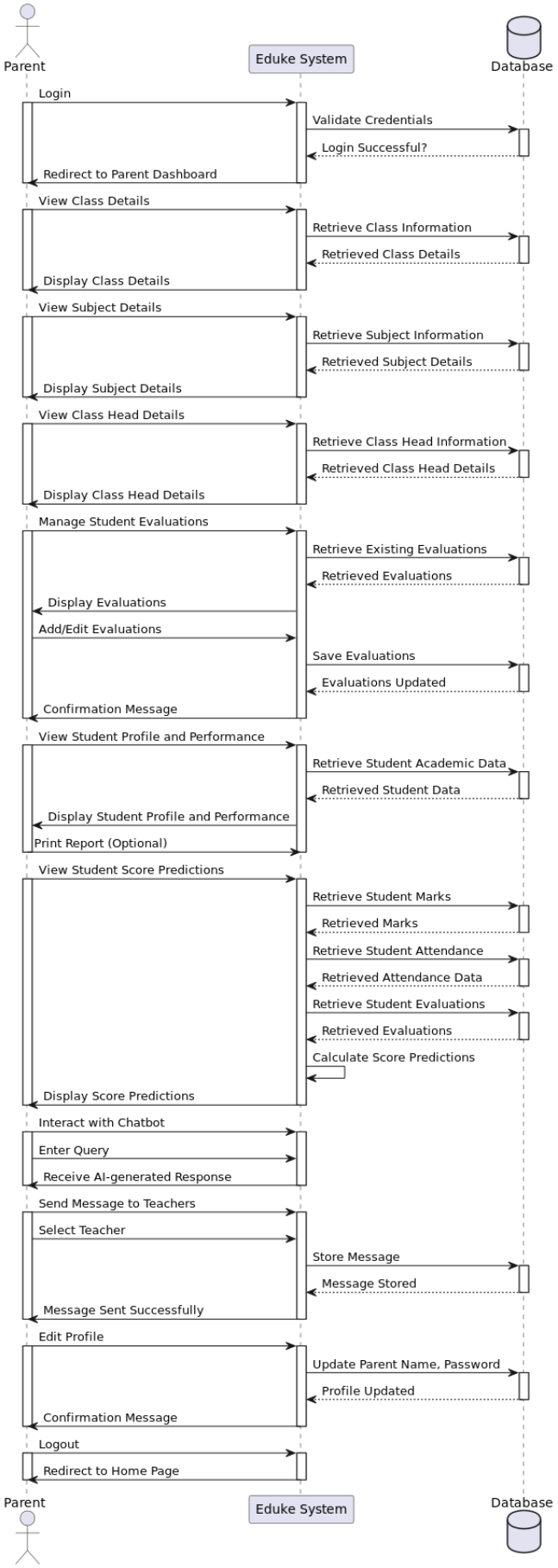
Subject Head



Student


Student
Eduke System
Database
Login
Validate Credentials
Login Successful?
Redirect to Student Dashboard
View Class Details
Retrieve Class Information
Retrieved Class Details
Display Class Details
View Subject Details
Retrieve Subject Information
Retrieved Subject Details
Display Subject Details
View Announcements and Study Materials
Retrieve Announcements and Study Materials
Retrieved Announcements and Study Materials
Display Announcements and Study Materials
Attend Quizzes
Retrieve Available Quizzes
Retrieved Quizzes
Display Quiz List
Select Quiz
Fetch Quiz Questions
Retrieved Quiz Questions
Display Questions
Answer Questions
Submit Quiz
Store Quiz Responses
Responses Saved
Calculate Score
Score Calculated
Display Quiz Score
View Student Profile and Performance
Retrieve Student Academic Data
Retrieved Student Data
Display Student Profile and Performance
Print Report (Optional)
View Student Score Predictions
Retrieve Student Marks
Retrieved Marks
Retrieve Student Attendance
Retrieved Attendance Data
Retrieve Student Evaluations
Retrieved Evaluations
Calculate Score Predictions
Display Score Predictions
Interact with Chatbot
Enter Query
Receive AI-generated Response
Send Message to Teachers
Select Teacher
Store Message
Message Stored
Message Sent Successfully
Edit Profile
Update Student Name, Password
Profile Updated
Confirmation Message
Logout
Redirect to Home Page
Student
Eduke System
Database

Parent


Parent
Eduke System
Database
Login
Validate Credentials
Login Successful?
Redirect to Parent Dashboard
View Class Details
Retrieve Class Information
Retrieved Class Details
Display Class Details
View Subject Details
Retrieve Subject Information
Retrieved Subject Details
Display Subject Details
View Class Head Details
Retrieve Class Head Information
Retrieved Class Head Details
Display Class Head Details
Manage Student Evaluations
Retrieve Existing Evaluations
Retrieved Evaluations
Display Evaluations
Add/Edit Evaluations
Save Evaluations
Evaluations Updated
Confirmation Message
View Student Profile and Performance
Retrieve Student Academic Data
Retrieved Student Data
Display Student Profile and Performance
Print Report (Optional)
View Student Score Predictions
Retrieve Student Marks
Retrieved Marks
Retrieve Student Attendance
Retrieved Attendance Data
Retrieve Student Evaluations
Retrieved Evaluations
Calculate Score Predictions
Display Score Predictions
Interact with Chatbot
Enter Query
Receive AI-generated Response
Send Message to Teachers
Select Teacher
Store Message
Message Stored
Message Sent Successfully
Edit Profile
Update Parent Name, Password
Profile Updated
Confirmation Message
Logout
Redirect to Home Page
Parent
Eduke System
Database

3.5 MODULAR DESIGN

3.5.1 STRUCTURE CHART

Structure charts are intended to give visual representations of the logical processes identified within the design. There are some variations to the exact notation used in structure charts, but all include an inverted tree structure with boxes showing each of the main logical actions.

In structure chart that each box represents a major item of the design. These items are also called modules and are another key aspect of structured design. Modules allow the system design to be split into a series of sub problems, each of which can be split in turn into sub modules and so on until the larger problem is solved when all the smaller sub modules are working correctly. The concept of modularization also allows work to be split between individuals in large teams, provided that they all agree on what each module does and how it can be used. Typically, modules in large software designs have data items passed to the module and then expect other data items to be passed back at the end of the module's operation.

Splitting the software design into modules also allows the modules to be tested separately to ensure that they work correctly, provided that they have a single exit and entry point to the module

In theory, should anything go wrong with the system or if it ever needed to be updated, the use of modules permits the person maintaining the system to know where each function is performed within the hierarchy of modules that make up the structure chart. It is not uncommon for the structured English or pseudo-code to be written in each of the module boxes that make up the structure chart. Whatever way the pseudo-code and structure chart are represented when combined, they should show the detailed design of the system.

3.5.2 MODULES DESCRIPTION

Admin

The Admin is responsible for overseeing the entire system. They manage institutions, classes, subjects, and student records. Admins assign class heads and subject heads, ensuring proper supervision of academic activities. They also have access to update institution details and monitor overall system performance.

Class Head

The Class Head is responsible for managing an entire class. They oversee student performance, attendance, and subject-related activities. They can view student profiles, monitor evaluations, and track attendance records. Class Heads play a crucial role in ensuring smooth communication between students, parents, and subject heads.

Subject Head

Subject Heads handle subject-specific tasks, including managing attendance, student evaluations, and marks. They can create and conduct quizzes, upload study materials, and make announcements for students. They also provide academic guidance, track student progress, and communicate with both students and parents.

Student

Students are the main users of Eduke, using the system to access their academic records, study materials, and quizzes. They can track their performance, view subject and class details, and receive important announcements. Eduke AI, the chatbot assistant, helps students by answering academic queries. Students can also communicate with subject heads and update their profile information.

Parent

Parents use Eduke to monitor their child’s academic progress. They can view their child’s attendance, marks, and score predictions. Parents can also submit student evaluations and interact with subject heads for better academic support. Additionally,

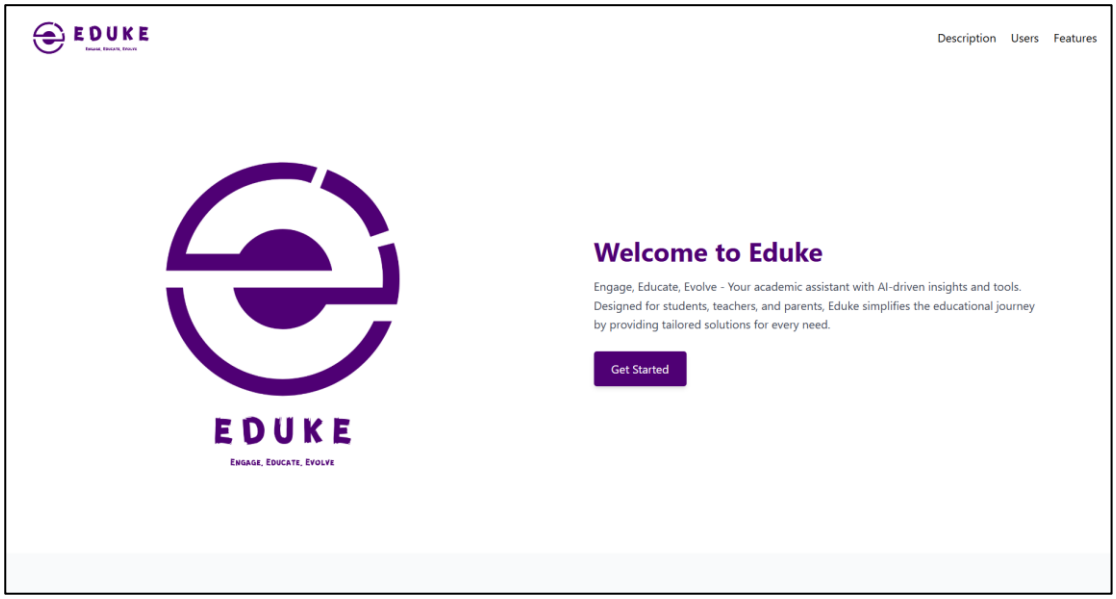
Eduke AI assists parents by providing insights and answering queries related to their child's performance.

3.6 INPUT DESIGN

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data into a usable form for processing data entry. The activity of putting data into the computer for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system.

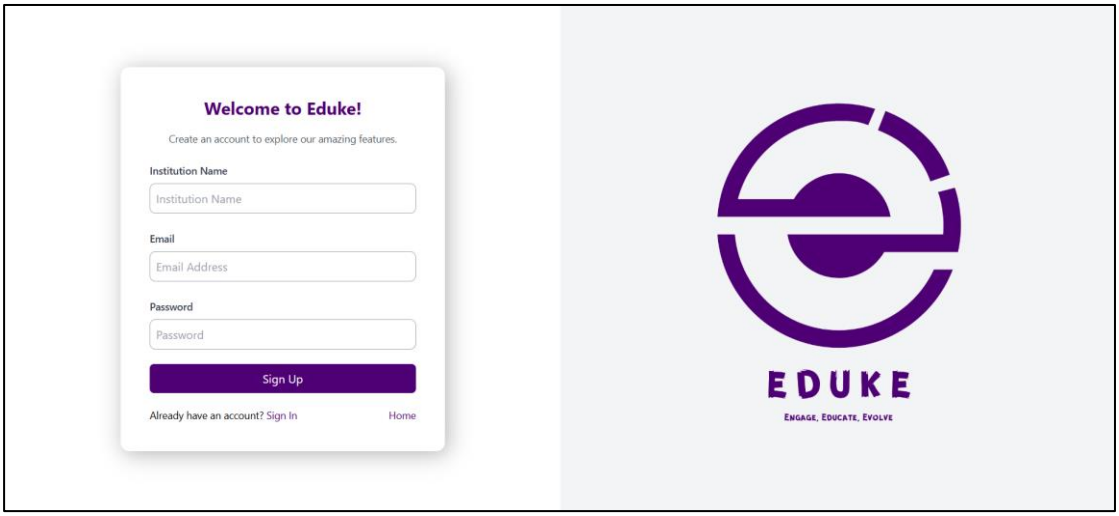
- What data should be given as input?
- The dialogue to guide the operating personnel in providing input.
- Methods for preparing input validations and steps to follow when error Occur

Home Page



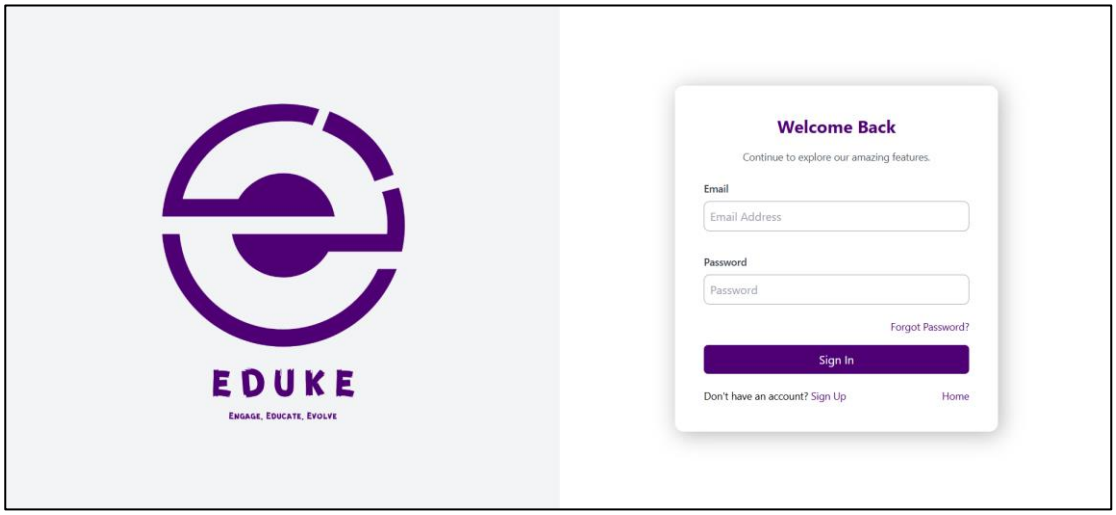
Admin Registration

The registration form contains two text fields, one for the user to enter an email and the other to enter the password. Once the details are entered, the Sign up button is clicked to register.



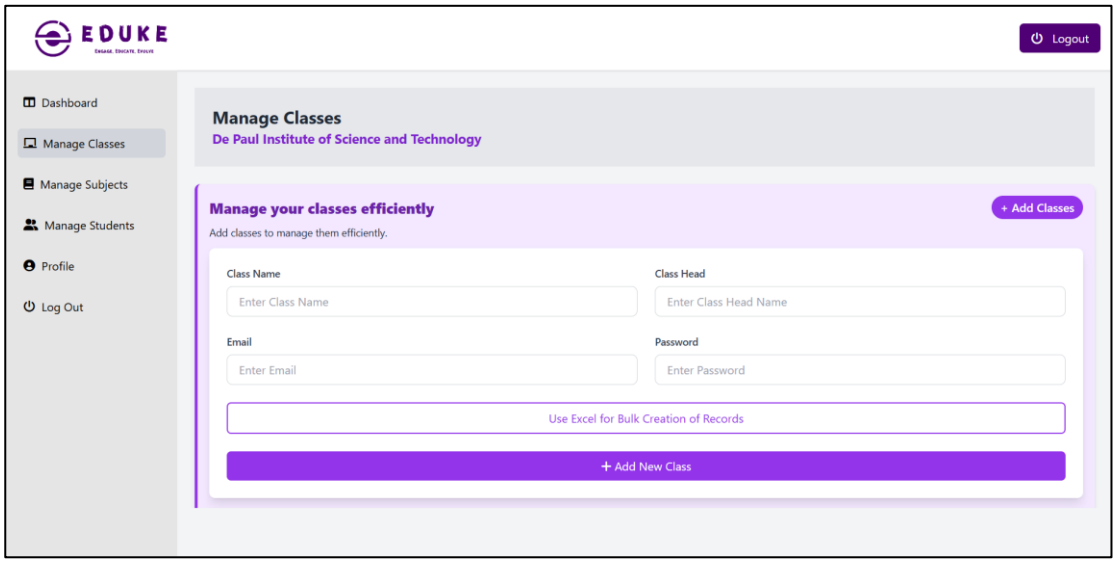
Admin Login

The login form contains two text fields, one for the user to enter an email and the other to enter the password. Once the details are entered, the Sign in button is clicked to login. Similarly, we have logins for class head, subject head, student and parent.

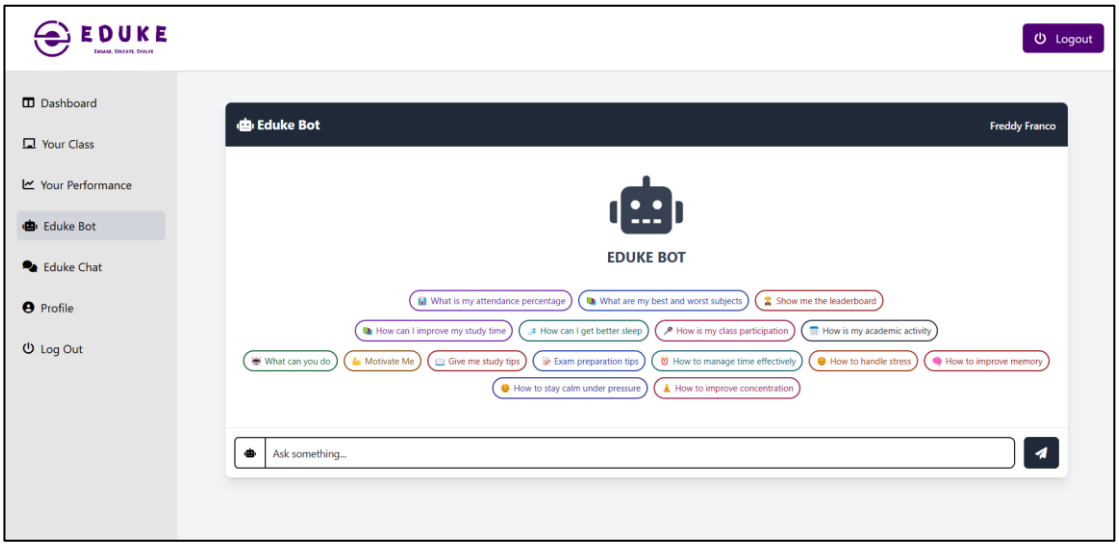


Admin Adding Classes

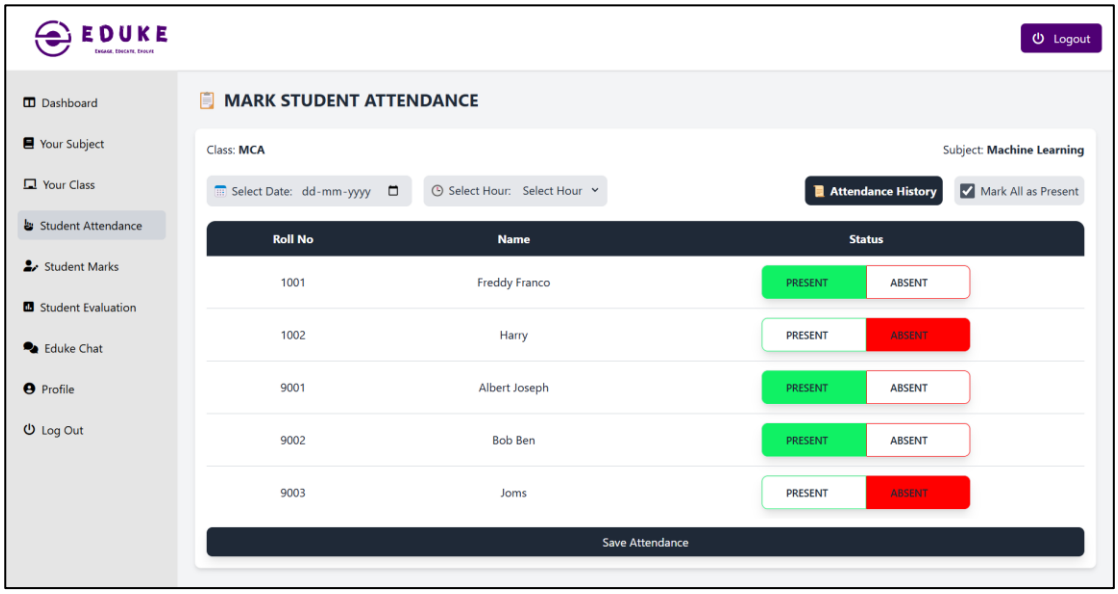
Similarly, admin can add edit, delete subjects, students, etc.



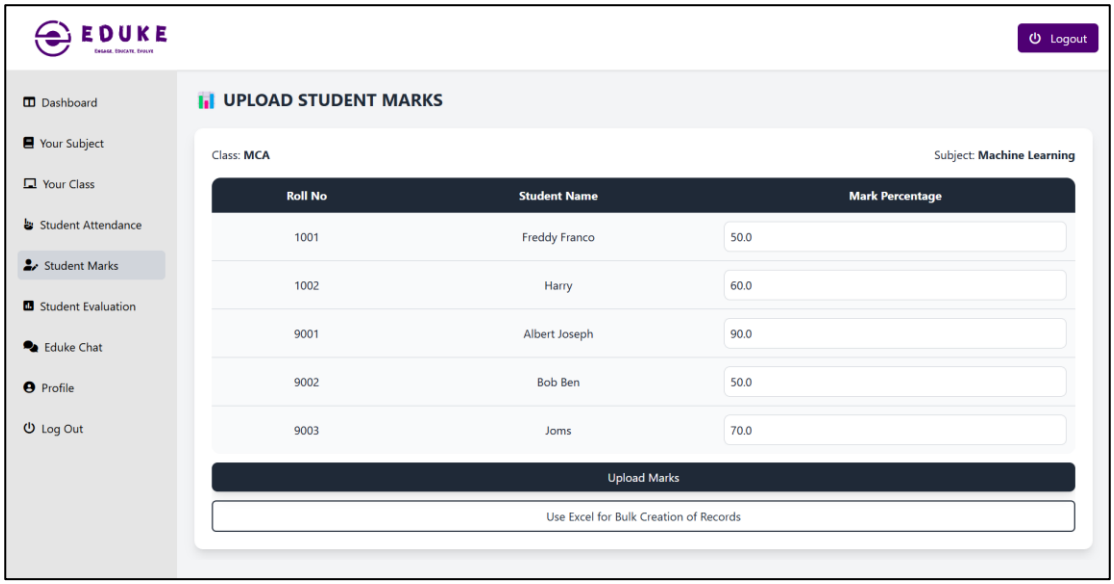
Chatbot page



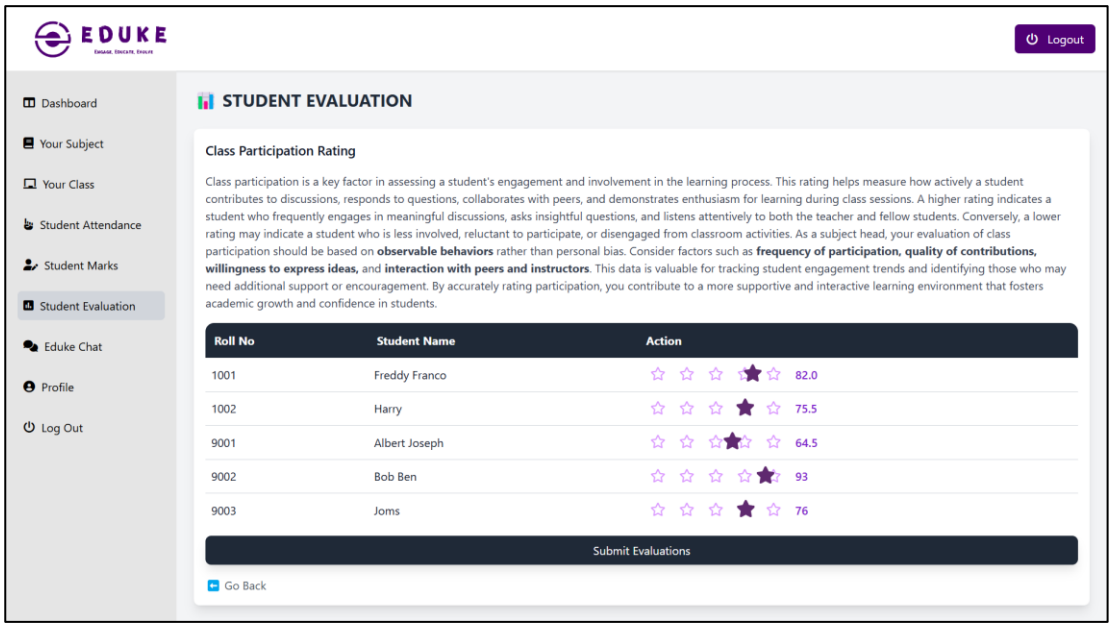
Attendance Page



Marks Page



Evaluations Page

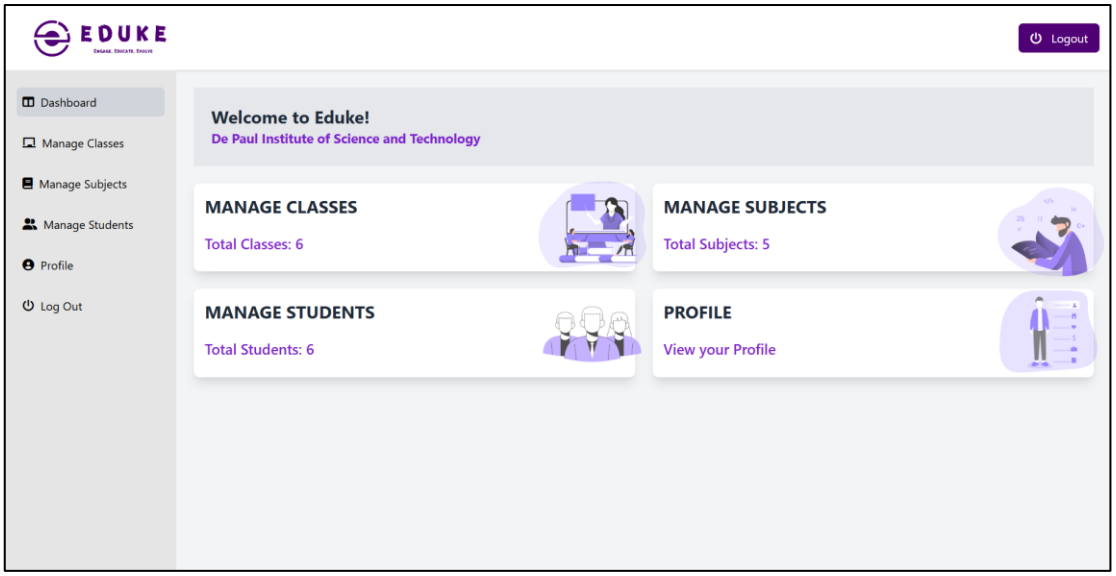


3.7 OUTPUT DESIGN

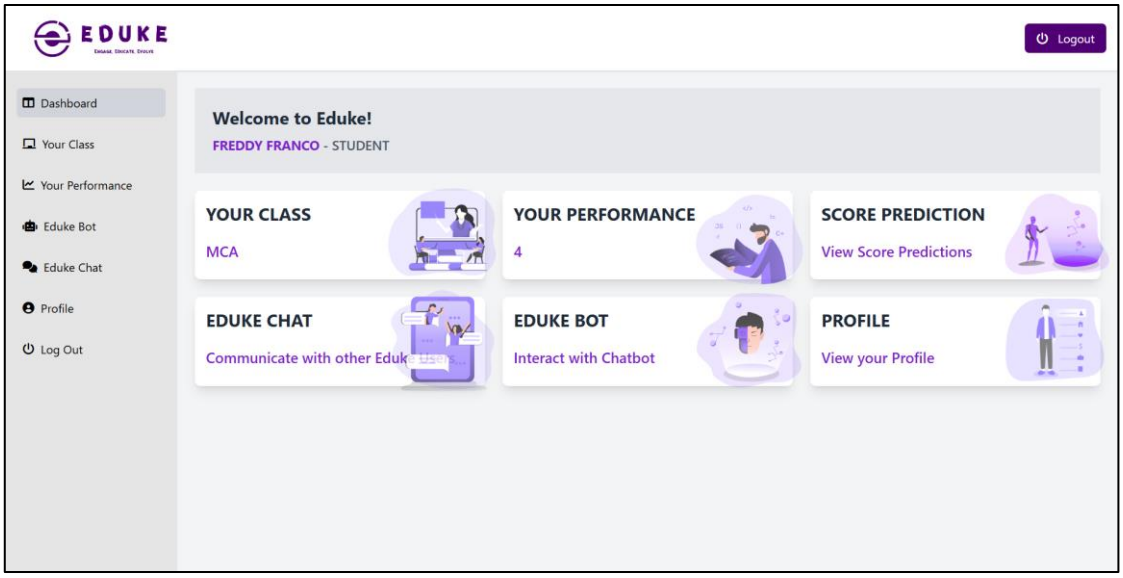
Computer output is the most important and direct information source to the user. Output design is a process that involves designing necessary outputs in the form of reports that should be given to the users according to the requirements. Efficient, intelligible output design should improve the system's relationship with the user and help in decision making. So, while designing output the following things are to be considered.

- Determine what information to present
- Arrange the presentation of information in an acceptable format
- Decide how to distribute the output to intended receipts

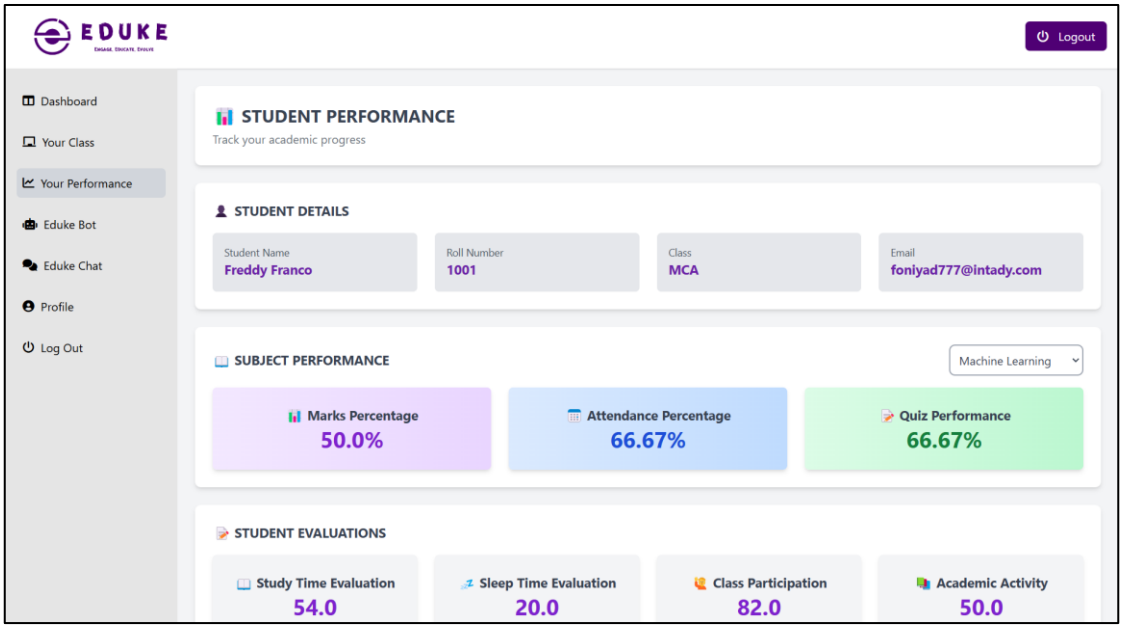
Admin Dashboard

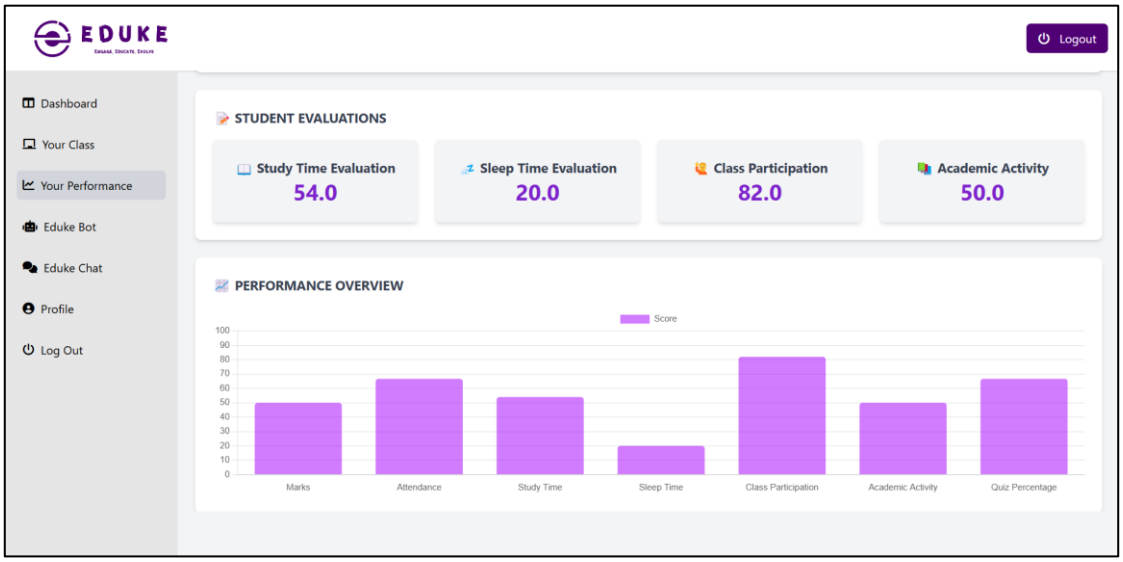


Student Dashboard

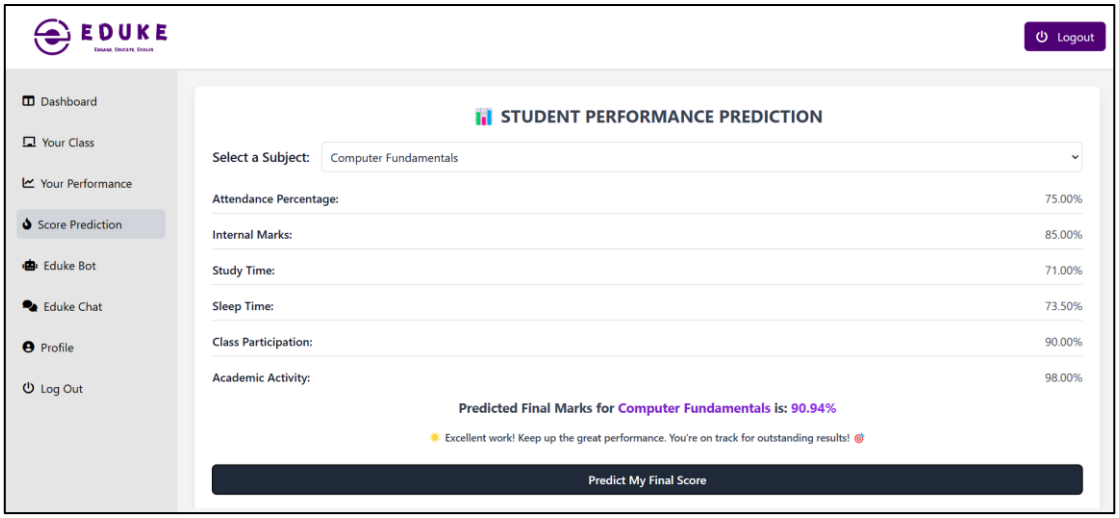


Student Performance Page





Score Prediction Page



SYSTEM ENVIRONMENT

4.1 INTRODUCTION TO SYSTEM ENVIRONMENT

The most common set of requirements defined by any operating system or software application is the physical computer resources, also known as hardware, A hardware requirements list is often accompanied by a hardware compatibility list (HCL), especially in case of operating systems. An HCL lists tested, compatible, and sometimes incompatible hardware devices for a particular operating system or application. The following subsections discuss the various aspects of hardware requirements.

Architecture

All computer operating systems are designed for a particular computer architecture. Most software applications are limited to particular operating systems running on particular architectures. Although architecture-independent operating systems and applications exist, most need to be recompiled to run on a new architecture. See also a list of common operating systems and their supporting architectures.

Processing power

The power of the central processing unit (CPU) is a fundamental system requirement for any software. Most software running on x86 architecture define processing power as the model and the clock speed of the CPU. Many other features of a CPU that influence its speed and power, like bus speed, cache, and MIPS are often ignored. This definition of power is often erroneous, as AMD Athlon and Intel Pentium CPUs at similar clock speed often have different throughput speeds. Intel Pentium CPUs have enjoyed a considerable degree of popularity, and are often mentioned in this category.

Memory

All software, when run, resides in the Random Access Memory (RAM) of a computer. Memory requirements are defined after considering demands of the application, operating system, supporting software and files, and other running processes. Optimal performance of other unrelated software running on a multi-tasking computer system is also considered when defining this requirement.

Secondary storage

Data storage device requirements vary, depending on the size of software installation, temporary files created and maintained while installing or running the software, and possible use of swap space (if RAM is insufficient).

Display adapter

Software requiring a better than average computer graphics display, like graphics editors and high-end games, often define high-end display adapters in the system requirements.

Peripherals

Some software applications need to make extensive and/or special use of some peripherals, demanding the higher performance or functionality of such peripherals. Such peripherals include CD-ROM drives, keyboards, pointing devices, network devices, etc.

Software requirements

Software requirements deal with defining software resource requirements and prerequisites that need to be installed on a computer to provide optimal functioning of an application. These requirements or prerequisites are generally not included in the software installation package and need to be installed separately before the software is installed.

Platform

A computing platform describes some sort of framework, either in hardware or software, which allows software to run. Typical platforms include a computer's architecture, operating system, or programming languages and their runtime libraries. Operating system is one of the requirements mentioned when defining system requirements (software). Software may not be compatible with different versions of same line of operating systems, although some measure of backward compatibility is often maintained. For example, most software designed for Microsoft Windows XP does not run on Microsoft Windows 98, although the converse is not always true. Similarly, software designed using newer features of Linux Kernel v2.6 generally

does not run or compile properly (or at all) on Linux distributions using Kernel v2.2 or v2.4.

APIs and drivers

Software making extensive use of special hardware devices, like high-end display adapters, needs special API or newer device drivers. A good example is DirectX, which is a collection of APIs for handling tasks related to multimedia, especially game programming, on Microsoft platforms.

Web browser

Most web applications and software depend heavily on web technologies to make use of the default browser installed on the system. Google Chrome, Brave, Microsoft Edge, etc are frequent choices of software running on Microsoft Windows, which makes use of ActiveX controls, despite their vulnerabilities.

4.2 SOFTWARE REQUIREMENTS SPECIFICATION

Database : MYSQL 5.1

Operating System : GNU Linux Derivatives/Windows 10/11

Front END : HTML, Tailwind CSS, JavaScript

Backend : Python (Django), MySQL

4.3 HARDWARE REQUIREMENTS SPECIFICATION

Processor : Intel® Core (TM) i3 CPU M 380 @ 2.53GHz

RAM : 4 GB RAM

Monitor : Standard color monitor

Keyboard : Standard Keyboard.

Mouse : Standard two button or higher

4.4 TOOLS, PLATFORMS

4.4.1 FRONT END TOOL

The front-end of an application is distinctly human. It's what the user sees, touches and experiences. In this respect, empathy is a required characteristic of a good frontend developer. The front-end of an application is less about code and more about how a user will interpret the interface into an experience. That experience can be the difference between a billion-dollar company and complete collapse. If you were a My space user in 2004, you were probably content with the experience. But once you started to use Facebook, you almost certainly had a better experience. You realized that you could socialize with a simpler design, no flashing banner ads, easy-to-find friends, etc. Facebook and My space had a lot of differences under the hood as well(back-end), but at least part of Facebook's triumph can be attributed to a better frontend and user experience.

The technologies used in front-end development commonly include:

HTML: All code in a web application is eventually translated to HTML. It's the language that web browsers understand and use to display information to users. A web developer's understanding of HTML is analogous to a carpenter's understanding of a screwdriver. It's so important and necessary that it's often assumed for employment.

Tailwind CSS: is a utility-first CSS framework that simplifies styling by providing a set of pre-defined utility classes. Unlike traditional CSS frameworks, it allows developers to build custom designs without writing custom CSS. Tailwind CSS is highly flexible, responsive, and efficient, enabling rapid development while maintaining a clean and scalable code structure. Its approach encourages reusability and consistency, making it a popular choice for modern web applications.

JavaScript: is a powerful, versatile programming language primarily used for creating interactive and dynamic web applications. As a core technology of the web alongside HTML and CSS, JavaScript enables features like real-time updates, animations, form validation, and event handling. It can be used for both frontend and backend development, thanks to frameworks and libraries like React, Vue, Node.js, and Express.

4.4.2 BACK-END TOOL

Web page, but if the application itself doesn't work, the application will be a failure. The back-end of an application is responsible for things like calculations, business logic, database interactions, and performance. Most of the code that is required to make an application work will be done on the back-end. Back-end code is run on the server, as opposed to the client. This means that back-end developers not only need to understand program. The back-end of a web application is an enabler for a front-end experience. An application's front-end may be the most beautifully crafted ming languages and databases, but they must have an understanding of server architecture as well. If an application is slow, crashes often, or constantly throws errors at users, it's likely because of back-end problems.

Django: Django is a high-level, open-source web framework written in Python that promotes rapid development and clean, pragmatic design. It follows the Model-View-Template (MVT) architecture, making it easy to build scalable and maintainable web applications. Django comes with built-in security features, an admin panel, and ORM for database management, reducing development time.

Features

- Provides a built-in authentication system for secure user management
- Includes an ORM to interact with databases like MySQL and PostgreSQL
- Supports scalability and rapid development with reusable components

MySQL: MySQL is an open-source relational database management system (RDBMS) that uses SQL (Structured Query Language) to manage and manipulate data. It is widely used for web applications and supports high-performance, scalable, and secure database management. MySQL is often integrated with programming languages like PHP, Python, and Java to build dynamic applications.

Key Capabilities of MySQL

- **Developer Productivity** – Provides an easy-to-use interface, automation, and built-in functions to enhance development efficiency.

- **Performance and Scalability** – Handles large datasets efficiently and supports replication, clustering, and partitioning for scalability.
- **Security and Reliability** – Offers robust security features like user authentication, encryption, and access controls to protect data.

4.4.3 OPERATING SYSTEM

WINDOWS 10

Windows 10 is a series of personal computer operating systems produced by Microsoft as part of its Windows NT family of operating systems. It is the successor to Windows 8.1, and was released to manufacturing on July 15, 2015, and broadly released for retail sale on July 29, 2015. Windows 10 receives new builds on an on going basis, which are available at no additional cost to users, in addition to additional test builds of Windows 10 which are available to Windows Insiders. Devices in enterprise environments can receive these updates at a slower pace, or use long-term support milestones that only receive critical updates, such as security patches, over their ten-year lifespan of extended support.

One of Windows 10's most notable features is support for universal apps, an expansion of the Metro-style apps first introduced in Windows 8. Universal apps can design to run across multiple Microsoft product families with nearly identical code— including PCs, tablets, smartphones, embedded systems, Xbox One, Surface Hub and Mixed Reality. The Windows user interface was revised to handle transitions between a mouse-oriented interface and a touchscreen-optimized interface based on available input devices— particularly on 2-in-1 PCs, both interfaces include an updated Start menu which incorporates elements of Windows 7's traditional Start menu with the tiles of Windows 8. Windows 10 also introduced the Microsoft Edge web browser, a virtual desktop system, a window and desktop management feature called Task View, support for fingerprint and face recognition login, new security features for enterprise environments, and DirectX 12.

Windows 10 received mostly positive reviews upon its original release in July 2015. Critics praised Microsoft's decision to provide a desktop-oriented interface in line

with previous versions of Windows, contrasting the tablet-oriented approach of 8, although Windows 10's touch-oriented user interface mode was criticized for containing regressions upon the touch-oriented interface of Windows 8. Critics also praised the improvements to Windows 10's bundled software over Windows 8.1, Xbox Live integration, as well as the functionality and capabilities of the Cortana personal assistant and the replacement of Internet Explorer with Edge. However, media outlets have been critical of changes to operating system behaviours, including mandatory update installation, privacy concerns over data collection performed by the OS for Microsoft and its partners and the adware-like tactics used to promote the operating system on its release.

SYSTEM IMPLEMENTATION

5.1 INTRODUCTION TO SYSTEM IMPLEMENTATION

A crucial phase in the system life cycle is the successful implementation of the new system design. Implementation simply means converting a new system design into operation. This involves creating computer compatible files, training, and telecommunication network before the system is up and running. A crucial factor in conversion is not disrupting the functioning of organization. Actual data were input into the program and the working of the system was closely monitored. It is a process of converting a new or revised system into an operational one. It is the essential stage in achieving a successful new system because usually it involves a lot of upheaval in the user. It must therefore be carefully planned and controlled to avoid problems. The implementation phase involves the following tasks:

- Careful planning.
- Investigation
- Design of methods
- Training of the staff in the changeover phase.
- Evaluation of changeover.

We implemented this new system in parallel run plan without making any disruptions to the on-going system, but only computerizing the whole system to make the work, evaluation and retrieval of data easier, faster and reliable.

TRAINING

System implementation is the process of making the newly designed system fully operational and consistent in performance. The logical miss-working the system can be identified if any. Various combinations of test data were feed. Each process accuracy/reliability checking was made. After the approval, the system was implemented in the user department. The preparation of implementation of documentation process is often viewed as total sum of the software documentation process. In a well-defined software development environment, however the presentation of implementation documents is essentially an interactive process that synthesis and recognizes document items that were produced during the analysis and design phase for the presentation to user.

CONVERSION METHODS

The implementation of the Eduke academic management system requires proper documentation to ensure a smooth transition and effective usage. There are three primary types of implementation documents:

Conversion Guide

The conversion guide outlines the necessary steps to transition the Eduke system from development to full operation. This phase includes tasks such as data migration, file conversion, and database setup to ensure that all student, class, and academic records are properly transferred into the system. It also details the requirements for creating new files and entering essential data into the platform. The conversion guide is crucial in minimizing errors and ensuring that the transition process does not disrupt the institution's operations.

User Guide

The user guide provides detailed instructions on how different users—such as students, parents, class heads, and subject heads—can navigate and utilize the system. It describes the functionalities available to each user role, including logging in, accessing student reports, managing attendance, entering marks, and interacting with the Eduke AI chatbot. Additionally, it defines the data processing methods used by the system, whether through offline storage or direct database access, ensuring users understand how to operate the platform effectively.

Operation Guide

The operation guide is designed for system administrators and technical support teams responsible for managing Eduke. It contains instructions on system initialization, database management, maintenance procedures, and troubleshooting common issues. The guide provides general information about the system, an overview of its functionalities, and a detailed description of how to run and monitor the platform. It also includes steps for updating the system, managing backups, and handling user access control to ensure data security and integrity.

By following these implementation documents, users and administrators can efficiently operate the Eduke system, ensuring a seamless and effective academic management experience.

5.2 CODING

Coding, sometimes called computer programming, is how we communicate with computers. Code tells a computer what actions to take, and writing code is like creating a set of instructions. By learning to write code, you can tell computers what to do or how to behave in a much faster way. You can use this skill to make websites and apps, process data, and do lots of other cool things.

5.2.1 CODING STANDARDS

Different modules specified in the design document are coded in the Coding phase according to the module specification. The main goal of the coding phase is to code from the design document prepared after the design phase through a high-level language and then to unit test this code.

Good software development organizations want their programmers to maintain to some well-defined and standard style of coding called coding standards. They usually make their own coding standards and guidelines depending on what suits their organization best and based on the types of software they develop. It is very important for the programmers to maintain the coding standards otherwise the code will be rejected during code review.

Purpose of Having Coding Standards:

- A coding standard gives a uniform appearance to the codes written by different engineers.
- It improves readability, and maintainability of the code and it reduces complexity also.
- It helps in code reuse and helps to detect error easily.
- It promotes sound programming practices and increases efficiency of the programmers.

Some of the coding standards are given below:

1. Limited use of global: These rules talk about which types of data that can be declared global and the data that can’t be.

2. Standard headers for different modules: For better understanding and maintenance of the code, the header of different modules should follow some standard format and information. The header format must contain below things that is being used in various companies:

   - Name of the module
   - Date of module creation
   - Author of the module
   - Modification history
   - Synopsis of the module about what the module does
   - Different functions supported in the module along with their input output parameters
   - Global variables accessed or modified by the module

3. Naming conventions for local variables, global variables, constants and functions:

   - Some of the naming conventions are given below:
   - Meaningful and understandable variables name help anyone to understand the reason of using it.
   - Local variables should be named using camel case lettering starting with small letter (e.g., local Data) whereas Global variables names.
   - Capital letter (e.g., Global Data). Constant names should be formed using capital letters only (e.g., CONSDATA).
   - It is better to avoid the use of digits in variable names.
   - The names of the function should be written in camel case starting with small letters.
   - The name of the function must describe the reason of using the function clearly and briefly.

1. Indentation:

Proper indentation is very important to increase the readability of the code. For making the code readable, programmers should use White spaces properly. Some of the spacing conventions are given below:

- There must be a space after giving a comma between two function arguments.
- Each nested block should be properly indented and spaced.
- Proper Indentation should be there at the beginning and at the end of each block in the program.
- All braces should start from a new line and the code following the end of braces also start from a new line.

2. Error return values and exception handling conventions:

All functions that encountering an error condition should either return a 0 or 1 for simplifying the debugging.

On the other hand, Coding guidelines give some general suggestions regarding the coding style that to be followed for the betterment of understand ability and readability of the code. Some of the coding guidelines are given below:

3. Avoid using a coding style that is too difficult to understand:

Code should be easily understandable. The complex code makes maintenance and debugging difficult and expensive.

4. Avoid using an identifier for multiple purposes:

Each variable should be given a descriptive and meaningful name indicating the reason behind using it. This is not possible if an identifier is used for multiple purposes and thus it can lead to confusion to the reader. Moreover, it leads to more difficulty during future enhancements.

1. Code should be well documented:

The code should be properly commented for understanding easily. Comments regarding the statements increase the understandability of the code.

2. Length of functions should not be very large:

Lengthy functions are very difficult to understand. That's why functions should be small enough to carry out small work and lengthy functions should be broken into small ones for completing small tasks.

5.2.2 SAMPLE CODES

```
def login_view(request):
    if request.method == 'POST':
        form = LoginForm(request.POST)
        if form.is_valid():
            email = form.cleaned_data['email']
            password = form.cleaned_data['password']

            # Authenticate the institution
            try:
                institution = Institution.objects.get(email=email, password=password)

                # Set session variables
                request.session['institution_id'] = institution.institution_id
                request.session['institution_name'] = institution.institution_name

                # Redirect to the admin dashboard
                return redirect('admin_dashboard')  # Adjust the URL name accordingly
            except Institution.DoesNotExist:
                messages.error(request, 'Invalid email or password.')
        else:
            messages.error(request, 'Please correct the errors below.')
    else:
        form = LoginForm()

    return render(request, 'login.html', {'form': form})

# Admin Logout
def logout(request):
    # Clear session and logout
    request.session.flush()
    return redirect('/')


def admin_classes(request):
    print(" View Loaded: admin_classes()")

    if 'institution_id' not in request.session:
        messages.error(request, 'Please log in to access this page.')
        print(" User not logged in. Redirecting to login.")
```

```
        return redirect('login')
    institution_id = request.session['institution_id']
    print(f" Institution ID: {institution_id}")

    try:
        institution = Institution.objects.get(institution_id=institution_id)
        print(f" Institution Found: {institution.institution_name}")
    except Institution.DoesNotExist:
        messages.error(request, "Institution not found.")
        print(" Institution not found. Redirecting to login.")
        return redirect('login')

    classes = Classes.objects.filter(institution_id=institution.institution_id)
    if request.method == 'POST':
        print(" POST Request Received.")
        print(f" Request Data: {request.POST}")

        print(" Handling New Class Addition")
        form = AddClassForm(request.POST)

        if form.is_valid():
            with transaction.atomic():
                class_name = form.cleaned_data['class_name']
                class_head = form.cleaned_data['class_head']
                email = form.cleaned_data['email']
                password = form.cleaned_data['password']

                print(f" New Class Data - Name: {class_name}, Head: {class_head}, Email:
{email}")

                user = Users.objects.create(role='class_head')
                print(" New User Created with ID:", user.id)

                new_class = Classes(
                    class_name=class_name,
                    class_head=class_head,
                    email=email,
                    password=password,
                    institution=institution,
                    user=user
                )
                new_class.save()


                print(f" New Class Added with ID: {new_class.id}. Data: Name:
{class_name}, Head: {class_head}, Email: {email}")

                send_account_creation_email(email, password, "class_head", class_head,
institution.email)
                print(" Account creation email sent.")
```

```
            messages.success(request, 'New class added successfully!')
            return redirect('admin_classes')
        else:
            messages.error(request, 'Error adding class. Please try again.')
            print(" Error: Form validation failed.")

    else:
        print(" Rendering Page with GET Request.")
        form = AddClassForm()

    context = {
        'institution_name': institution.institution_name,
        'classes': classes,
        'form': form,
    }

    print(f" Sending context to template: {context}")
    print(" Rendering admin/admin_classes.html")
    return render(request, 'admin/admin_classes.html', context)


def admin_subjects(request):
    # Check if the user is logged in
    if 'institution_id' not in request.session:
        messages.error(request, 'Please log in to access this page.')
        return redirect('login')

    # Get the institution_id from the session
    institution_id = request.session['institution_id']
    print(f"DEBUG: Institution ID from session - {institution_id}")

    try:
        # Fetch the institution based on the session institution_id
        institution = Institution.objects.get(institution_id=institution_id)
        print(f"DEBUG: Institution found - {institution.institution_name}")
    except Institution.DoesNotExist:
        messages.error(request, "Institution not found.")
        return redirect('login')

    # Fetch all classes for the dropdown
    classes = Classes.objects.filter(institution_id=institution_id).values('id', 'class_name')
    print(f"DEBUG: Classes retrieved - {list(classes)}")

    # Fetch subjects along with class names
    subjects = []
    with connection.cursor() as cursor:
        cursor.execute("""
            SELECT
                ms.id, ms.subject_name, ms.subject_head, ms.email,
                c.class_name
```

```
            FROM main_subjects ms
            JOIN main_classes c ON ms.class_obj_id = c.id
            WHERE c.institution_id = %s
        """, [institution_id])
        subjects = cursor.fetchall()

    print(f"DEBUG: Retrieved subjects - {subjects}")

    if request.method == "POST":
        form = AddSubjectForm(request.POST)
        if form.is_valid():
            subject_name = form.cleaned_data['subject_name']
            subject_head = form.cleaned_data['subject_head']
            email = form.cleaned_data['email']
            password = form.cleaned_data['password']
            class_id = form.cleaned_data['class_id'].id  # Get class ID

            with connection.cursor() as cursor:
                try:
                    # Step 1: Insert into main_users table
                    cursor.execute("INSERT INTO main_users (role) VALUES (%s)",
['subject_head'])
                    user_id = cursor.lastrowid  # Get the last inserted user ID

                    if not user_id:
                        raise Exception("User ID not retrieved after insert!")

                    print(f"DEBUG: Inserted into main_users - User ID: {user_id}")

                    # Step 2: Insert into main_subjects table
                    cursor.execute("""
                        INSERT INTO main_subjects (subject_name, subject_head, email,
password, user_id, class_obj_id)
                        VALUES (%s, %s, %s, %s, %s, %s)
                    """, [subject_name, subject_head, email, password, user_id, class_id])

                    print(f"DEBUG: Inserted into main_subjects - Subject: {subject_name},
Class ID: {class_id}")

                    # **Step 3: Send Account Creation Email**
                    send_account_creation_email(email, password, "subject_head",
subject_head, institution.email)

                    messages.success(request, "Subject added successfully!")
                    return redirect('admin_subjects')

                except Exception as e:
                    print(f"ERROR: {e}")
                    messages.error(request, f"An error occurred: {e}")
        else:
            print("DEBUG: Form is invalid")
```

```
    else:
        form = AddSubjectForm()

    context = {
        'institution_name': institution.institution_name,
        'form': form,
        'classes': classes,
        'subjects': subjects  # Pass subjects data to template
    }

    return render(request, 'admin/admin_subjects.html', context)


def admin_students(request):
    # Check if the user is logged in
    if 'institution_id' not in request.session:
        messages.error(request, 'Please log in to access this page.')
        return redirect('login')

    # Get the institution_id from the session
    institution_id = request.session['institution_id']
    print(f"DEBUG: Institution ID from session - {institution_id}")

    try:
        # Fetch the institution based on the session institution_id
        institution = Institution.objects.get(institution_id=institution_id)
        print(f"DEBUG: Institution found - {institution.institution_name}")
    except Institution.DoesNotExist:
        messages.error(request, "Institution not found.")
        return redirect('login')

    # Fetch all classes for the dropdown
    classes = Classes.objects.filter(institution_id=institution_id)
    print(f"DEBUG: Classes retrieved - {list(classes.values('id', 'class_name'))}")

    # Fetch students along with class name and class head name
    with connection.cursor() as cursor:
        cursor.execute("""
            SELECT
                s.id, s.name AS student_name, s.roll_no, s.email, c.class_name,
                c.class_head
            FROM main_students s
            LEFT JOIN main_classes c ON s.class_obj_id = c.id
            WHERE c.institution_id = %s
        """, [institution_id])
        students = cursor.fetchall()

    print(f"DEBUG: Students retrieved - {students}")

    # Handle form submission
```

```
    if request.method == "POST":
        form = AddStudentForm(request.POST)

        if form.is_valid():
            name = form.cleaned_data['name']
            roll_no = int(form.cleaned_data['roll_no'])  # Ensure integer conversion
            email = form.cleaned_data['email']  # Extract email
            password = form.cleaned_data['password']
            class_obj_id = form.cleaned_data['class_id'].id  # Extracting the integer ID

            print(f"DEBUG: Form Data - Name: {name}, Roll No: {roll_no}, Email:
{email}, Password: {password}, Class ID: {class_obj_id}")

            try:
                with transaction.atomic():
                    with connection.cursor() as cursor:
                        # Insert into users table (Student)
                        cursor.execute("INSERT INTO main_users (role) VALUES ('student')")
                        student_user_id = cursor.lastrowid
                        print(f"DEBUG: Inserted into users table (student), User ID:
{student_user_id}")

                        # Insert student record (Including Email)
                        cursor.execute("""
                            INSERT INTO main_students (name, roll_no, email, password,
class_obj_id, user_id)
                            VALUES (%s, %s, %s, %s, %s, %s)
                        """, [name, roll_no, email, password, class_obj_id, student_user_id])
                        student_id = cursor.lastrowid
                        print(f"DEBUG: Inserted into students table, Student ID: {student_id}")

                        # Insert into users table (Parent)
                        cursor.execute("INSERT INTO main_users (role) VALUES ('parent')")
                        parent_user_id = cursor.lastrowid
                        print(f"DEBUG: Inserted into users table (parent), User ID:
{parent_user_id}")

                        # Insert parent record
                        cursor.execute("""
                            INSERT INTO main_parents (student_id, password, name, user_id)
                            VALUES (%s, %s, NULL, %s)
                        """, [student_id, password, parent_user_id])
                        print(f"DEBUG: Inserted into parents table, Parent User ID:
{parent_user_id}")

                    # Send account creation email to student
                    send_account_creation_email(email, password, "student", name,
institution.email)
                    messages.success(request, "Student and Parent added successfully!")
```

```
                return redirect('admin_students')

            except Exception as e:
                print(f"ERROR: {e}")
                messages.error(request, f"Error: {str(e)}")
        else:
            print("DEBUG: Form validation failed")
            messages.error(request, "Failed to add student. Please check the form.")
    else:
        form = AddStudentForm()

    context = {
        'institution_name': institution.institution_name,
        'form': form,
        'students': students,
        'classes': classes,
    }

    return render(request, 'admin/admin_students.html', context)


def admin_student_performance(request, student_id):
    if 'institution_id' not in request.session:
        messages.error(request, 'Please log in to access this page.')
        return redirect('login')

    institution_id = request.session['institution_id']
    print(f"DEBUG: Institution ID from session - {institution_id}")

    print(f"Received student_id: {student_id}, Type: {type(student_id)}")

    # Validate student_id
    if not student_id or not str(student_id).isdigit():
        print("Invalid student_id, redirecting to admin_students.")
        messages.error(request, "Invalid student ID.")
        return redirect('admin_students')

    try:
        with connection.cursor() as cursor:
            # Fetch student details (excluding parent details)
            cursor.execute("""
                SELECT
                    s.id, s.roll_no, s.name, s.email, s.class_obj_id, c.class_name
                FROM main_students s
                JOIN main_classes c ON s.class_obj_id = c.id
                WHERE s.id = %s
            """, [student_id])
            student_row = cursor.fetchone()

            if not student_row:
                print("Student not found, redirecting to admin_students.")
```

```
            messages.error(request, "Student not found.")
            return redirect('admin_students')

        student = {
            'id': student_row[0],
            'roll_no': student_row[1],
            'name': student_row[2],
            'email': student_row[3],
            'class_id': student_row[4],
            'class_name': student_row[5],
        }
        print(f"Student Name: {student['name']}")

    # Extract subject_id from GET request
    subject_id = request.GET.get('subject_id', None)

    # Fetch subjects for the student's class
    subjects = []
    if student.get("class_id"):
        with connection.cursor() as cursor:
            cursor.execute("""
                SELECT id, subject_name FROM main_subjects WHERE class_obj_id =
%s
            """, [student["class_id"]])
            subjects = [{"id": row[0], "name": row[1]} for row in cursor.fetchall()]

    # Get the name of the selected subject
    selected_subject_name = None
    if subject_id:
        selected_subject_name = next((s["name"] for s in subjects if str(s["id"]) ==
subject_id), None)
        print(f"[DEBUG] Selected Subject: {selected_subject_name} (ID:
{subject_id})")

    # Initialize default evaluation data
    evaluations = {
        "marks_percentage": 0,
        "attendance_percentage": 0,
        "study_time_rating": 0,
        "sleep_time_rating": 0,
        "class_participation_rating": 0,
        "academic_activity_rating": 0,
    }

    # Fetch student evaluation details for the selected subject
    if subject_id:
        print(f"[DEBUG] Fetching evaluation data for subject:
{selected_subject_name} (ID: {subject_id})...")
        with connection.cursor() as cursor:
            cursor.execute("""
                SELECT marks_percentage, attendance_percentage,
```

```
                    study_time_rating, sleep_time_rating,
                    class_participation_rating, academic_activity_rating
                FROM main_studentevaluation
                WHERE student_id = %s AND subject_id = %s
            """, [student_id, subject_id])
            row = cursor.fetchone()

            if row:
                evaluations = {
                    "marks_percentage": round(row[0] or 0, 2),
                    "attendance_percentage": round(row[1] or 0, 2),
                    "study_time_rating": round(row[2] or 0, 2),
                    "sleep_time_rating": round(row[3] or 0, 2),
                    "class_participation_rating": round(row[4] or 0, 2),
                    "academic_activity_rating": round(row[5] or 0, 2),
                }
                print(f"[DEBUG] Evaluation Data: {evaluations}")
            else:
                print(f"[WARNING] No evaluation data found for
{selected_subject_name}.")

        # Fetch quiz performance for the selected subject
        quiz_percentage = 0
        if subject_id:
            print(f"[DEBUG] Fetching quiz performance for {selected_subject_name} (ID:
{subject_id})...")
            with connection.cursor() as cursor:
                cursor.execute("""
                    SELECT COUNT(*) AS total_attempted,
                        SUM(CASE WHEN q.correct_option = r.student_response THEN 1
ELSE 0 END) AS correct_answers
                    FROM main_quizresponse r
                    JOIN main_quizquestions q ON r.question_id = q.id
                    JOIN main_quizzes mq ON q.quiz_id = mq.id
                    WHERE r.student_id = %s AND mq.subject_id = %s
                """, [student_id, subject_id])
                row = cursor.fetchone()

                if row:
                    total_attempted, correct_answers = row[0], row[1] or 0
                    quiz_percentage = round((correct_answers / total_attempted) * 100, 2) if
total_attempted > 0 else 0
                    print(f"[DEBUG] Quiz Performance: Attempted = {total_attempted},
Correct = {correct_answers}, Percentage = {quiz_percentage}%")
                else:
                    print(f"[WARNING] No quiz data found for {selected_subject_name}.")

        # Prepare data for Graph
        graph_data = [
            float(evaluations.get("marks_percentage", 0)),
            float(evaluations.get("attendance_percentage", 0)),
```

```
            float(evaluations.get("study_time_rating", 0)),
            float(evaluations.get("sleep_time_rating", 0)),
            float(evaluations.get("class_participation_rating", 0)),
            float(evaluations.get("academic_activity_rating", 0)),
            float(quiz_percentage)
        ]

    except Exception as e:
        print(f"[ERROR] An error occurred: {e}")
        return redirect('error_page')  # Redirect to an error page if needed

    # Prepare context for the template
    context = {
        'student': student,
        'subjects': subjects,
        'selected_subject_id': subject_id,
        'evaluations': evaluations,
        'quiz_percentage': quiz_percentage,
        "graph_data": json.dumps(graph_data)
    }

    return render(request, 'admin/admin_student_performance.html', context)

def subject_head_marks(request):
    subject_id = request.session.get('subject_id')
    if not subject_id:
        return redirect('subject_head_login')

    subject_data = None
    student_list = []

    try:
        with connection.cursor() as cursor:
            # Fetch subject details along with class name
            cursor.execute("""
                SELECT s.id, s.subject_name, c.class_name, s.class_obj_id
                FROM main_subjects s
                JOIN main_classes c ON s.class_obj_id = c.id
                WHERE s.id = %s
            """, [subject_id])
            subject = cursor.fetchone()

            if subject:
                subject_data = {
                    'id': subject[0],
                    'subject_name': subject[1],
                    'class_obj_name': subject[2],  # Class name
                    'class_obj_id': subject[3],
                }

                # Handle form submission to update marks
```

```
            if request.method == "POST":
                success = True  # Track success

                for key, value in request.POST.items():
                    if key.startswith("marks_"):
                        student_id = key.split("_")[1]
                        mark_value = value.strip()

                        try:
                            mark_value = float(mark_value)  # Convert to float safely

                            #  Step 1: Check if the record exists in `main_marks`
                            cursor.execute("""
                                SELECT id FROM main_marks WHERE student_id = %s AND
subject_id = %s
                            """, [student_id, subject_data['id']])
                            mark_record = cursor.fetchone()

                            if mark_record:
                                #  Step 2: If exists, UPDATE `main_marks`
                                cursor.execute("""
                                    UPDATE main_marks
                                    SET mark_percentage = %s
                                    WHERE student_id = %s AND subject_id = %s
                                """, [mark_value, student_id, subject_data['id']])
                            else:
                                #  Step 3: If not exists, INSERT into `main_marks`
                                cursor.execute("""
                                    INSERT INTO main_marks (mark_percentage, student_id,
subject_id)
                                    VALUES (%s, %s, %s)
                                """, [mark_value, student_id, subject_data['id']])

                            #  Step 4: Check if the record exists in `main_studentevaluation`
                            cursor.execute("""
                                SELECT id FROM main_studentevaluation
                                WHERE student_id = %s AND subject_id = %s
                            """, [student_id, subject_data['id']])
                            evaluation = cursor.fetchone()

                            if evaluation:
                                #  Step 5: If exists, UPDATE `main_studentevaluation`
                                cursor.execute("""
                                    UPDATE main_studentevaluation
                                    SET marks_percentage = %s
                                    WHERE student_id = %s AND subject_id = %s
                                """, [mark_value, student_id, subject_data['id']])
                            else:
                                #  Step 6: If not exists, INSERT into `main_studentevaluation`

                                cursor.execute("""
```

```
                                INSERT INTO main_studentevaluation (student_id,
subject_id, marks_percentage)
                                VALUES (%s, %s, %s)
                            """, [student_id, subject_data['id'], mark_value])

                    except ValueError:  # Catch invalid input
                        print(f"Invalid mark value for student {student_id}: {value}")
                        success = False

            # Display success or error message
            if success:
                messages.success(request, " Marks updated successfully!",
extra_tags='marks_success')
            else:
                messages.error(request, " Failed to update marks. Please try again.",
extra_tags='marks_error')

            # Fetch students of this class
            cursor.execute("SELECT id, roll_no, name FROM main_students WHERE
class_obj_id = %s", [subject_data['class_obj_id']])
            students = cursor.fetchall()

            for student in students:
                student_id = student[0]

                # Fetch marks for each student
                cursor.execute("""
                    SELECT mark_percentage FROM main_marks
                    WHERE student_id = %s AND subject_id = %s
                """, [student_id, subject_data['id']])
                mark = cursor.fetchone()

                student_list.append({
                    'id': student_id,
                    'roll_no': student[1],
                    'name': student[2],
                    'mark_percentage': mark[0] if mark else None
                })

    except Exception as e:
        print(f"Database error: {e}")
        messages.error(request, "An unexpected error occurred.",
extra_tags='marks_error')

    return render(request, 'subject_head/subject_head_marks.html', {
        'subject': subject_data,
        'students': student_list
    })
```

```
def subject_head_attendance(request):
    subject_id = request.session.get('subject_id')

    if not subject_id:
        print("Subject ID not found in session. Redirecting to login.")
        return redirect('subject_head_login')

    with connection.cursor() as cursor:
        # Fetch subject and class details
        cursor.execute("""
            SELECT s.subject_name, c.class_name
            FROM main_subjects s
            JOIN main_classes c ON s.class_obj_id = c.id
            WHERE s.id = %s
        """, [subject_id])
        subject = cursor.fetchone()

        if not subject:
            print(f"No subject found for subject_id: {subject_id}. Redirecting to
dashboard.")
            return redirect('subject_head_dashboard')

        subject_name, class_name = subject
        print(f" Subject: {subject_name}, Class: {class_name}")

        # Fetch students of the subject's class
        cursor.execute("""
            SELECT s.id, s.roll_no, s.name
            FROM main_students s
            WHERE s.class_obj_id = (SELECT class_obj_id FROM main_subjects
WHERE id = %s)
        """, [subject_id])
        students = cursor.fetchall()
        print(f"Students Fetched ({len(students)} students): {students}")

        # Handle POST request (Attendance Submission)
        if request.method == "POST":
            print("Received Attendance Submission")

            attendance_date = request.POST.get('attendance_date')
            hour = request.POST.get('hour')

            if not attendance_date or not hour:
                print("Missing Date or Hour. Showing error message.")
                messages.error(request, "Please select both date and hour.")
                return redirect('subject_head_attendance')

            hour = int(hour)  # Convert hour to integer

            # **Check if attendance already exists for this date & hour**
```

```
        cursor.execute("""
            SELECT COUNT(*) FROM main_attendance
            WHERE subject_id = %s AND attendance_date = %s AND hour = %s
        """, [subject_id, attendance_date, hour])
        existing_records = cursor.fetchone()[0]

        if existing_records > 0:
            print(f"Attendance for {attendance_date}, Hour {hour} already exists!")
            messages.warning(request, f"Attendance for {attendance_date}, Hour {hour} has already been recorded.")
            return redirect('subject_head_attendance')

        # Insert new attendance records
        for student in students:
            student_id = student[0]
            status = request.POST.get(f'attendance_{student_id}', 'absent')  # Default to 'absent' if not checked

            cursor.execute("""
                INSERT INTO main_attendance (student_id, subject_id, attendance_date, hour, status, created_at)
                VALUES (%s, %s, %s, %s, %s, NOW())
            """, [student_id, subject_id, attendance_date, hour, status])
            print(f" Attendance Added - Student: {student_id}, Date: {attendance_date}, Hour: {hour}, Status: {status}")

            # **Calculate Attendance Percentage**
            cursor.execute("""
                SELECT COUNT(*) FROM main_attendance
                WHERE student_id = %s AND subject_id = %s
            """, [student_id, subject_id])
            total_classes = cursor.fetchone()[0]

            cursor.execute("""
                SELECT COUNT(*) FROM main_attendance
                WHERE student_id = %s AND subject_id = %s AND status = 'present'
            """, [student_id, subject_id])
            present_count = cursor.fetchone()[0]

            attendance_percentage = (present_count / total_classes) * 100 if total_classes > 0 else 0
            print(f"Student {student_id} - Attendance: {attendance_percentage:.2f}%")

            # **Update or Insert into StudentEvaluation**
            cursor.execute("""
                SELECT id FROM main_studentevaluation
                WHERE student_id = %s AND subject_id = %s
            """, [student_id, subject_id])
            existing_record = cursor.fetchone()
```

```
            if existing_record:
             cursor.execute("""
                  UPDATE main_studentevaluation
                  SET attendance_percentage = %s
                  WHERE student_id = %s AND subject_id = %s
               """, [attendance_percentage, student_id, subject_id])
               print(f" Updated Attendance Percentage: {attendance_percentage:.2f}% for
Student {student_id}")
            else:
               cursor.execute("""
                  INSERT INTO main_studentevaluation (student_id, subject_id,
attendance_percentage)
                  VALUES (%s, %s, %s)
               """, [student_id, subject_id, attendance_percentage])
               print(f" Inserted New Attendance Percentage:
{attendance_percentage:.2f}% for Student {student_id}")

         messages.success(request, " Attendance successfully recorded!")
         return redirect('subject_head_attendance')

   context = {
      'subject_name': subject_name,
      'class_name': class_name,
      'students': students,
   }

   return render(request, 'subject_head/subject_head_attendance.html', context)


import pandas as pd
import numpy as np
import joblib
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Load dataset
df = pd.read_csv('ml/Eduke_FInal_training_Data.csv')

# Separate features and target variable
X = df.drop(columns=['Final Marks'])
y = df['Final Marks']

# Apply Min-Max Scaling (Normalize features between 0 and 1)
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)
```

```
# Split data
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
random_state=42)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

joblib.dump(model, 'ml/final_model.pkl')
joblib.dump(scaler, 'ml/scaler.pkl')

# Evaluate model
y_pred = model.predict(X_test)

print("Model Performance:")
print("Mean Absolute Error (MAE):", round(mean_absolute_error(y_test, y_pred), 2))
print("Mean Squared Error (MSE):", round(mean_squared_error(y_test, y_pred), 2))
print("R² Score:", round(r2_score(y_test, y_pred), 2))
print("Model training complete and saved as 'ml/final_model.pkl'.")
```

5.3 DEBUGGING

Debugging is the process of detecting and removing of existing and potential errors (also called as ‗bugs„) in a software code that can cause it to behave unexpectedly or crash. To prevent incorrect operation of a software or system, debugging is used to find and resolve bugs or defects. When various subsystems or modules are tightly coupled, debugging becomes harder as any change in one module may cause more bugs to appear in another. Sometimes it takes more time to debug a program than to code it. To debug a program, user has to start with a problem, isolate the source code of the problem, and then fix it. A user of a program must know how to fix the problem as knowledge about problem analysis is expected. When the bug is fixed, then the software is ready to use. Debugging tools (called debuggers) are used to identify coding errors at various development stages. They are used to reproduce the conditions in which error has occurred, then examine the program state at that time and locate the cause. Programmers can trace the program execution step-by-step by evaluating the value of variables and stop the execution wherever required to get the value of variables or reset the program variables. Some programming language packages provide a debugger for checking the code for errors while it is being written at run time.

5.4 UNIT TESTING

Unit testing in the Eduke academic management system is a crucial step in ensuring the reliability and correctness of individual components within the software. By testing smaller units, such as functions, modules, or database queries, developers can identify and fix potential issues before they affect the overall system. This process helps maintain the quality and performance of Eduke, making it a more stable and efficient platform.

One of the key areas for unit testing is student login and authentication. Tests are conducted to verify that students can log in successfully using valid credentials while also ensuring that incorrect login attempts trigger appropriate error messages. This helps prevent unauthorized access while providing a seamless login experience for users.

Another critical aspect of testing involves marks entry and retrieval. Subject heads need to be able to input student marks accurately, and students should be able to view their results correctly. Unit tests validate that the marks are stored and retrieved correctly from the database, ensuring transparency and accuracy in academic performance tracking.

Attendance management is also an essential feature that undergoes rigorous unit testing. The system must correctly record student attendance, update records in real time, and allow parents and students to access attendance history without errors. These tests confirm that attendance data is properly managed and displayed within the dashboard.

Eduke Bot, the chatbot assistant, requires extensive testing to ensure it understands student queries and provides relevant responses. Unit tests check its ability to process different variations of a question, handle spelling mistakes, and generate meaningful answers based on the user's needs. This ensures that students receive helpful and accurate academic assistance.

In addition to academic tracking, Eduke also includes student performance predictions based on historical data such as marks, attendance, and evaluations. Unit tests validate the accuracy of these predictions, ensuring that the system provides reliable insights into student progress.

For parents, the Eduke dashboard allows them to access student profiles, track academic progress, and generate reports. Unit testing ensures that parents can retrieve information without issues and that the generated reports reflect accurate and up-to-date data. Similarly, the messaging system within Eduke undergoes testing to confirm that parents can communicate with class heads or subject heads effectively, with messages being sent and received without delays.

SYSTEM PLANNING AND SCHEDULING

6.1 INTRODUCTION TO SYSTEM PLANNING AND SCHEDULING

Planning and scheduling are distinct but inseparable aspects of managing the successful project. The process of planning primarily deals with selecting the appropriate policies and procedures in order to achieve the objectives of the project. Scheduling converts the project action plans for scope, time cost and quality into an operating timetable. The translating of the project criteria for scope, time, cost, and quality and the requirements for human resources, communications, risk and procurement into workable―machinery‖ for the project team a critical interface juncture for the project team. Taken together with the project plan and budget, the schedule becomes the major tool for the management of projects. In addition, the integrated cost-time schedule serves as the fundamental basis for monitoring and controlling project activity throughout its life cycle.

This basic level paper addresses the integrated processes of planning and scheduling of multifaced/multidisciplinary programs. The paper presents a working level summary of the major Project Management topics involved in the planning process. The paper also details a systematic process for transforming the Project Plan into the Schedule and the use of the Project Schedule as a model for project control. Intended for the project management novice, the paper concludes with a suggested professional development scheme.

6.1.1 PLANNING A SOFTWARE PROJECT

The basic project planning steps that every project manager needs to know can be broken down as parts of the first two phases of project management: Initiation and Planning. While those phases give a broad outline of what should be happening at different stages of a project’s lifecycle, they don’t provide much of a clear picture of how to go about your project planning.

6.1.1.1 STEPS INVOLVED IN PLANNING A SYSTEM

- Create and Analyze Business Case
- Identify and Meet Stakeholders for Approval

- Define Project Scope
- Set Project Goals and Objectives
- Determine Project Deliverables
- Create Project Schedule and Milestones
- Assignment of Tasks
- Carry Out Risk Assessment

Project planning doesn't have to be difficult or cause any nervous stress since the beginning of every project is basically the same. You can follow the same set project planning steps and hone them through experience of every project you are involved with.

Breaking down the steps:

1.How to Create and Analyze Business Case?

The business case is the reason why your organization needs to carry out the project. It should outline the problem, such as a lack of repeat customers or a day longer supply line than competitors and describe how this will be solved and how much monetary benefit should accrue to the organization once the project is completed.

2. How to Identify and Meet Relevant Stakeholders for Approval?

Identifying project stakeholders means listing anyone who will be affected by your project, so includes the public and government regulatory agencies. For the project planning phase however, it should only be necessary to meet those who will directly decide whether the project will happen or not.

3. Define Project Scope?

The scope of your project is an outline of what it is and isn't setting out to achieve. It is necessary to delineate the boundaries of your project to prevent ―scope creep‖, i.e., your resources going towards something that's not in your project's goals.

4.Set Goals and Objectives

The goals and objectives for your project will build on the initial objectives outlined in the business plans. At this step you will give finer detail to the initial broad ideas and set them in a project charter as reference points for your project as it proceeds.

5. Determine Deliverables

Deliverables are the concrete results that your project produces. One of the most important project planning steps is to decide on what these deliverables will be and who is responsible for both producing and receiving them.

6. Create Project Schedule and Milestones

Your project schedule is a very important document that outlines when different tasks of a project are due to begin and end, along with major measurement milestones. It will be referred to when measuring project progress. It will be available to all stakeholders and should be adhered to as closely as possible.

7. Assignment of Tasks

Within your team everyone should know what their role is and who is responsible for different elements of the project. Assigning tasks clearly should remove any uncertainty about roles and responsibilities on your team.

8. Carry Out Risk Assessment

Having a functional risk management plan means performing a strong assessment at the planning stage of the project. All potential risks should be identified along with their possible effect on the project and likelihood of occurring.

SYSTEM COST ESTIMATION

7.1 INTRODUCTION

A project can only come together with all the necessary materials and labour, and those materials and labours cost money. Putting together a budget that keeps costs to a minimum, while maximizing the project's quality and scope can be challenging. This is why proper cost estimation is important.

Cost estimation in project management is the process of forecasting the financial and other resources needed to complete a project within a defined scope. Cost estimation accounts for each element required for the project—from materials to labour—and calculates a total amount that determines a project's budget. An initial cost estimate can determine whether an organization greenlights a project, and if the project moves forward, the estimate can be a factor in defining the project's scope. If the cost estimation comes in too high, an organization may decide to pare down the project to fit what they can afford (it is also required to begin securing funding for the project). Once the project is in motion, the cost estimate is used to manage all of its affiliated costs in order to keep the project on budget.

7.2 LOC BASED ESTIMATION

A line of code (LOC) is any line of text in a code that is not a comment or blank line, and also header lines, in any case of the number of statements or fragments of statements on the line. LOC clearly consists of all lines containing the declaration of any variable, and executable and non-executable statements. As Lines of Code (LOC) only counts the volume of code, you can only use it to compare or estimate projects that use the same language and are coded using the same coding standards.

Features:

- They are used in assessing a project's performance or efficiency.
- Variations such as ―source lines of code‖, are used to set out a codebase.
- LOC is frequently used in some kinds of arguments.

Advantages:

- Most used metric in cost estimation.

Disadvantages:

- Very difficult to estimate the LOC of the final program from the
- It correlates poorly with quality and efficiency of code.
- It doesn't consider complexity.

FUNCTIONAL POINT BASED ESTIMATION

Function Point Analysis (FPA) is a method or set of rules of Functional Size Measurement. It assesses the functionality delivered to its users, based on the user's external view of the functional requirements. It measures the logical view of an application, not the physically implemented view or the internal technical view.

The Function Point Analysis technique is used to analyses the functionality delivered by software and Unadjusted Function Point (UFP) is the unit of measurement.

Objectives of FPA:

- The objective of FPA is to measure the functionality that the user requests and receives.
- The objective of FPA is to measure software development and maintenance independently of the technology used for implementation.
- It should be simple enough to minimize the overhead of the measurement process.
- It should be a consistent measure among various projects and organizations.

SYSTEM TESTING

8.1 INTRODUCTION TO SYSTEM TESTING

Software testing is an investigation conducted to provide stakeholders with information about the quality of the product or service under test. Software testing can also provide an objective, independent view of the software to allow the business to appreciate and understand the risks of software implementation. Test techniques include the process of executing a program or application with the intent of finding software bugs (errors or other defects).

Software testing involves the execution of a software component or system component to evaluate one or more properties of interest. In general, these properties indicate the extent to which the component or system under test:

- Meets the requirements that guided its design and development.
- Responds correctly to all kinds of inputs.
- Performs its functions within an acceptable time.
- It is sufficiently usable.
- It can be installed and run in its intended environments
- Achieves the general result its stakeholder's desire.

Defects and Failures

Not all software defects are caused by coding errors. One common source of expensive defects is requirement gaps, e.g., unrecognized requirements which results in errors of omission by the program designer.

Testing levels

There are generally four recognized levels of tests: unit testing, integration tests, component interface testing, and system testing. Tests are frequently grouped by where they are added in the software development process, or by the level of specificity of the test.

8.2 INTEGRATION TESTING

It tests the integration of each module in the system. It also tests to find discrepancies between the system and its original objective. In this testing analysis we are trying to find areas where modules have been designed with different specification for data length, type etc. In my project integration testing is performed after unit testing. In unit testing it ensure that each module is working properly. In integration testing ensure that complete system working properly.

In this project we ensure that the admin side and customer side together work properly. First of all, the admin logs in to the system and enters all necessary details such as item, special item, price etc. These details are entered so that it is helpful for the users. Then the customer logs in to the system and enters their personal details, email & password. They can order their food item for the events. Then the admin logs in to the system and go through the order details and so on.

8.3 VALIDATION TESTING

Validation refers to the process of using software in a live environment in order to find errors. During the course of validating the system, failure may occur and sometimes the coding has to be changed according to the requirement.

- Password entry check
  i. Ensure that the user must enter a valid password.
- Numeric and character check to corresponding fields
  ii. Ensures that the user must enter the numeric and character value in the specified field.
- Required Field Validator
  iii. Ensures that the user does not skip an entry.
- Date Check
  iv. Ensures that it is valid date or not.

8.3.1 TEST CASES

A test case is a set of sequential steps to execute a test operating on a set of predefined inputs to produce certain expected outputs. There are two types of test cases: - manual

and automated. Manual test case is executed manually while an automated test case is executed using automation.

TEST CASE FOR LOGIN:

Form	**Input Data**	**Expected Result**	**Actual Result**	**Remark**
Entry Form	Index Page for the software	Registration and login	Must be logged in	Success
Login Form	Enter valid user id and password	Should validate user and provide link to user account	Got entry to accounts	Success

VALIDATION TESTING

Form	**Input Data**	**Expected Result**	**Actual Result**	**Remark**
Try to Add details	When no details entered	When the valid details entered	An error message the field is required	Success
Try to Add details	On clicking check button	Entered information are not valid	An error message please enter a valid data	Success
Try to add details	On clicking submit button	Displays a message detail are added successfully	Displays a message detail are added successfully	Success

INTEGATION TESTING

Form	Expected Result	Actual Result	Remark
Login and user account forms	Get entry to Appropriate user page	Appropriate user page is displayed	Success
Main Page and other forms	Logout from all other forms should lead to main page	Main page is displayed	Success
Register Customer	Check all mandatory fields and validate all entered data fields	If any error found display message and the same screen displayed else record saved and confirmed	Success

WHITE BOX TESTING

White box sometimes called ―Glass box testing‖ is a test case design uses the control structure of the procedural design to drive test case. Using white box testing methods, the following tests were made on the system.

a) All independent paths within a module have been exercised once. In our system, ensuring that case was selected and executed checked all case structures. The bugs that were prevailing in some part of the code where fixed.

b) All logical decisions were checked for the truth and falsity of the values.

c) In the login pages check the credentials (valid user).

d) In the dashboard page opens up only after proper registration.

e) Once the account is deleted the same user login should not be permitted

BLACK BOX TESTING

Black box testing focuses on the functional requirements of the software. This is black box testing enables the software engineering to derive a set of input conditions that will fully exercise all functional requirements for a program. Black box testing is not an alternative to white box testing rather it is complementary approach that is likely to uncover a different class of errors that white box methods like,

1. Interface errors.

2. Performance in data structure.

3. Performance errors.

4. Initializing and termination errors.

Black box testing ensure that the system satisfies its functionality. The main functions are working without error. And ensure that system working correctly in the initialization there is proper authentication in admin and customer side also check and verify the termination or logout works properly.

SYSTEM MAINTENANCE

9.1. INTRODUCTION FOR SYSTEM MAINTENANCE

Maintenance is making adaptation of the software for external changes (requirements changes or enhancements) and internal changes (fixing bugs). When changes are made during the maintenance phase all preceding steps of the model must be revisited.

9.2 MAINTENANCE

Maintaining the Eduke academic management system is essential to ensure its long-term reliability, efficiency, and adaptability to changing user needs. The maintenance process includes three key types:

Corrective Maintenance: Corrective maintenance focuses on identifying and fixing bugs, errors, or system malfunctions that may arise during the operation of Eduke. This includes resolving issues related to incorrect data processing, login failures, broken links, or performance bottlenecks. By addressing these faults promptly, the system remains stable and functional for students, teachers, and administrators.

Adaptive Maintenance: Adaptive maintenance involves modifying the Eduke system to accommodate changes in the technological environment, institutional requirements, or regulatory policies. This may include updates to ensure compatibility with newer web browsers, integrating new database technologies, or adjusting features based on feedback from users. Minor adaptive updates can be incorporated into routine maintenance, while significant modifications may require dedicated development efforts.

Perfective Maintenance: Perfective maintenance aims at improving the system's efficiency, performance, and user experience without altering its core functionality. Enhancements may include optimizing database queries for faster access, refining the Eduke AI chatbot for better responses, improving the UI design for a more intuitive experience, or adding new analytics features for better academic tracking. The goal is to enhance the system's usability while ensuring it continues to meet institutional needs effectively.

By implementing a structured maintenance plan, Eduke can evolve to meet the dynamic requirements of students, parents, and educators, ensuring a smooth and efficient academic management experience.

SYSTEM SECURITY MEASURES

10.1 INTRODUCTION TO SYSTEM SECURITY MEASURES

The security of a computer system is a crucial task. It is a process of ensuring the confidentiality and integrity of the OS. Security is one of most important as well as the major task in order to keep all the threats or other malicious tasks or attacks or program away from the computer's software system. A system is said to be secure if its resources are used and accessed as intended under all the circumstances, but no system can guarantee absolute security from several of various malicious threats and unauthorized access.

The security of a system can be threatened via two violations:

Threat: A program that has the potential to cause serious damage to the system.

Attack: An attempt to break security and make unauthorized use of an asset.

Security violations affecting the system can be categorized as malicious and accidental threats. Malicious threats, as the name suggests are a kind of harmful computer code or web script designed to create system vulnerabilities leading to back doors and security breaches. Accidental Threats, on the other hand, are comparatively easier to be protected against. Example: Denial of Service DDos attack.

Security can be compromised via any of the breaches mentioned:

Breach of confidentiality: This type of violation involves the unauthorized reading of data.

Breach of integrity: This violation involves unauthorized modification of data.

Breach of availability: It involves unauthorized destruction of data.

Theft of service: It involves the unauthorized use of resources.

Denial of service: It involves preventing legitimate use of the system. As mentioned before, such attacks can be accidental in nature.

10.2 OPERATING SYSTEM LEVEL SECURITY

Operating system security (OS security) is the process of ensuring OS integrity, confidentiality and availability. OS security refers to specified steps or measures used

to protect the OS from threats, viruses, worms, malware or remote hacker intrusions. OS security encompasses all preventive-control techniques, which safeguard any computer assets capable of being stolen, edited or deleted if OS security is compromised. OS security encompasses many different techniques and methods which ensure safety from threats and attacks. OS security allows different applications and programs to perform required tasks and stop unauthorized interference. OS security may be approached in many ways, including adherence to the following:

- Performing regular OS patch updates
- Installing updated antivirus engines and software
- Scrutinizing all incoming and outgoing network traffic through a firewall Creating secure accounts with required privileges only (i.e., user management)

10.3 DATABASE LEVEL SECURITY

Database security refers to the various measures organizations take to ensure their databases are protected from internal and external threats. Database security includes protecting the database itself, the data it contains, its database management system, and the various applications that access it. Organizations must secure databases from deliberate attacks such as cyber security threats, as well as the misuse of data and databases from those who can access them. In the last several years, the number of data breaches has risen considerably. In addition to the considerable damage these threats pose to a company's reputation and customer base, there are an increasing number of regulations and penalties for data breaches that organizations must deal with, such as those in the General Data Protection Regulation (GDPR)—some of which are extremely costly. Effective database security is key for remaining compliant, protecting organizations,, reputations, and keeping their customers. Security concerns for internet-based attacks are some of the most persistent challenges to database security. Hackers devise new ways to infiltrate databases and steal data almost daily. Organizations must ensure their database security measures are strong enough to withstand these attacks. Some of these cyber security threats can be difficult to detect, like phishing scams in which user credentials are compromised and used without permission. Malware and ransomware are also common cyber security threats.

Another critical challenge for database security is making sure employees, partners, and contractors with database access don't abuse their credentials. These exfiltration vulnerabilities are difficult to guard against because users with legitimate access can take data for their own purposes. Edward Snowdens compromise of the NSA is the best example of this challenge. Organizations must also make sure users with legitimate access to database systems and applications are only privy to the information they need for work. Otherwise, there's greater potential for them to compromise database security.

There are three layers of database security: the database level, the access level, and the perimeter level. Security at the database level occurs within the database itself, where the data live. Access layer security focuses on controlling who is allowed to access certain data or systems containing it. Database security at the perimeter level determines who can and cannot get into databases. Each level requires unique security solutions.

10.4 SYSTEM-LEVEL SECURITY

System-level security refers to the architecture, policy and processes that ensure data and system security on individual computer systems. It facilitates the security of standalone and/or network computer systems/servers from events and processes that can exploit or violate its security or stature.

System-level security is part of a multi-layered security approach in which information security (IS) is implemented on an IT infrastructure's different components, layers or levels. System-level security is typically implemented on end-user computer and server nodes. It ensures that system access is granted only to legitimate and trusted individuals and applications. The key objective behind system-level security is to keep system secure, regardless of security policies and processes at other levels. If other layers or levels are breached, the system must have the ability to protect itself. Methods used to implement system-level security are user/ID login credentials, antivirus and system-level firewall applications

FUTURE ENHANCEMENT & SCOPE OF FURTHER DEVELOPMENT

11.1 INTRODUCTION

As education becomes increasingly digital, the Eduke system stands as an innovative platform designed to enhance student learning, streamline academic management, and foster seamless communication between students, parents, and educators. With its structured approach to academic performance prediction and personalized assistance through AI, Eduke has laid a strong foundation for modernizing education management.

While the current implementation provides essential features such as academic performance tracking, AI-driven chatbot support, and online assessments, there is immense potential for further development. Future enhancements could include machine learning-driven adaptive learning, deeper AI-based academic guidance, integration with virtual classrooms, and real-time collaboration tools. Additionally, expanding Eduke AI to support voice-based interactions and multilingual capabilities can make the system more inclusive.

The future scope of Eduke is vast, with opportunities to integrate blockchain-based credential verification, cloud-based storage solutions, and data-driven student counseling systems. By continuously evolving with emerging technologies and educational trends, Eduke can become a more dynamic and intelligent academic companion, ensuring personalized learning experiences and improved academic outcomes.

11.2 MERITS OF THE SYSTEM

- Enhanced academic outcomes through proactive performance prediction and personalized guidance.
- Streamlined academic management for educators and administrators, ensuring efficient handling of classes, subjects, and student records.
- Empowered students with real-time access to performance data, study materials, and personalized academic insights.
- Increased parental engagement by providing transparent monitoring of student progress and performance reports.

- Improved communication among teachers, class heads, subject heads, and parents via integrated messaging features.
- Scalability to support a growing user base and evolving educational requirements.
- Continuous system enhancement through user feedback and comprehensive data analytics.

11.3 LIMITATIONS OF THE SYSTEM

- Dependency on internet access, making it inaccessible to students and parents in areas with poor connectivity.
- Possible technical issues or server downtimes that may disrupt access to the platform.
- Unresponsive user interface on certain devices, causing usability issues for students, teachers, and parents.
- Limited AI accuracy in predicting student performance, as predictions rely on available historical data.
- Challenges in integrating the system with external educational tools or third-party platforms.
- Potential user resistance to adapting to digital academic management, especially among traditional educators.
- The need for continuous updates and improvements to ensure system efficiency and relevance.
- Dependence on accurate data entry by educators and administrators for reliable performance analysis.

11.4 FUTURE ENHANCEMENT OF THE SYSTEM

The future enhancements of the Eduke system hold significant potential to further improve the academic experience for students, parents, and educators. With the continuous advancement of technology and educational methodologies, several key areas can be explored to make the platform more efficient, user-friendly, and impactful.

1. **AI-Powered Personalized Learning** – Integrating AI-driven learning recommendations based on students' performance and study patterns can help create personalized study plans, improving learning outcomes.
2. **Enhanced Chatbot Capabilities** – Expanding Eduke AI to handle more complex academic queries, provide detailed explanations, and support voice-based interactions can make the system more interactive and accessible.
3. **Mobile App Development** – Creating a dedicated mobile application will improve accessibility, allowing students and parents to monitor academic progress, receive notifications, and interact with the system anytime, anywhere.
4. **Gamification for Student Engagement** – Implementing gamification features like quizzes, leaderboards, and achievement badges can increase student engagement and motivation in learning.
5. **Integration with External Learning Platforms** – Linking Eduke with platforms like Coursera, Khan Academy, or YouTube educational content can provide additional learning resources for students.
6. **Data Analytics for Educators** – Providing teachers and class heads with advanced analytics and predictive insights on student performance can enable better academic planning and early intervention for struggling students.
7. **Biometric or Two-Factor Authentication** – Enhancing security by implementing biometric login or two-factor authentication can help protect sensitive student and academic data.
8. **Offline Mode Functionality** – Introducing an offline mode where students can access stored materials and submit assignments once they reconnect can be beneficial for those with limited internet access.

9. **Parental Engagement Features** – Adding real-time notifications and progress reports can improve parent involvement in their child's academic journey.
10. **Scalability for Multi-Institution Use** – Expanding the platform to support multiple institutions and different academic structures can make Eduke a widely adopted system for educational management.

By embracing these enhancements, Eduke can continue evolving into a comprehensive academic platform, providing an innovative, data-driven, and personalized learning experience while supporting institutions in managing academic performance more efficiently.

REFERENCES

12.1 REFERENCES

- Django Software Foundation. (n.d.). *Django Documentation*. Retrieved from https://docs.djangoproject.com/
- GeeksforGeeks. (n.d.). *Introduction to Django*. Retrieved from https://www.geeksforgeeks.org/django-tutorial/
- W3Schools. (n.d.). *Django Tutorial*. Retrieved from https://www.w3schools.com/django/
- Mozilla Developer Network (MDN). (n.d.). *JavaScript Guide*. Retrieved from https://developer.mozilla.org/en-US/docs/Web/JavaScript
- Tailwind CSS. (n.d.). *Tailwind Documentation*. Retrieved from https://tailwindcss.com/docs
- Oracle. (n.d.). *MySQL Documentation*. Retrieved from https://dev.mysql.com/doc/
- W3Schools. (n.d.). *MySQL Tutorial*. Retrieved from https://www.w3schools.com/sql/
- GeeksforGeeks. (n.d.). *Introduction to MySQL*. Retrieved from https://www.geeksforgeeks.org/introduction-to-mysql/
- Stanford NLP Group. (n.d.). *NLP Overview*. Retrieved from https://nlp.stanford.edu/
- GeeksforGeeks. (n.d.). *Introduction to NLP*. Retrieved from https://www.geeksforgeeks.org/natural-language-processing-nlp-tutorial/
- Towards Data Science. (n.d.). *How to Build a Chatbot Using NLP*. Retrieved from https://towardsdatascience.com/how-to-build-a-chatbot-using-natural-language-processing-5020ead4e244